

Tracking Daily Inflation in Cambodia

The Plain-Language Guide to How We Measure Daily Prices, Clean Data, and Calculate the Consumer Price Index

What is this book about?

This handbook explains in simple, clear, everyday words how our automated system tracks prices across Cambodia every day. It shows how we collect over 45,000 prices every morning, fix typos, convert US Dollars to Khmer Riel, calculate fair inflation numbers, and predict where prices are heading weeks before official monthly reports come out.

No heavy technical jargon. Just clear explanations, practical everyday examples, and simple math.

Written by: Cambodia Inflation Intelligence Team

Target Country: Kingdom of Cambodia

Edition: Plain-Language Master Edition (September 2026)

Contents

1	Why Cambodia Needs a Daily Price Tracker	4
1.1	Cambodia Uses Two Currencies at the Same Time	4
1.2	What Cambodian Families Spend Most of Their Money On	4
1.3	Why the Traditional Monthly Survey is Too Slow	4
1.4	Our Daily Solution: Automated Web Intelligence	5
2	The 3-Step Journey of Our Data (Bronze, Silver, Gold)	6
2.1	The 3 Refining Steps (The Medallion Approach)	6
2.2	How Airflow and Cosmos Keep Everything Running on Time	6
2.3	Splitting Big Tables into Monthly Folders	6
3	Collecting the Prices Every Morning (Bronze Layer)	8
3.1	The Stores and Services We Actually Scrape	8
3.2	Master Inventory: Stores and Available Items	8
3.3	How We Scrape Each Source (Methods and Technology)	10
3.4	Visiting Websites Politely Without Getting Blocked	12
3.5	Never Recording the Same Price Twice (Deduplication)	13
3.6	The Ingestion Circuit Breaker and Dual-Currency Protection	13
4	Cleaning the Data and Text Normalization (Silver Layer)	14
4.1	The Khmer and English Language Challenge	14
4.2	Text Cleaning and Unit Normalization Rules	14
5	AI Vector Embeddings: Matching Items and Sorting Products	15
5.1	What is a Vector Embedding?	15
5.2	Part 1: Semantic Product Matching (Connecting Scraped Names to Catalog)	15
5.2.1	The Mathematical Cosine Similarity	15
5.2.2	The 3 Matching Tiers and Decision Rules	15
5.2.3	Crucial Spec Guardrails (Preventing False Merges)	15
5.3	Part 2: AI-First Product Classification into UN COICOP Baskets	16
6	A Real-Life Example: The Journey of a Bottle of Fish Sauce	18
7	The Rules of Fair Price Math	19
7.1	Why Simple Averages Lie (The Egg Example)	19
7.2	The Fair Way: The Jevons Geometric Mean	19
7.3	Why People Buy Chicken When Pork Gets Expensive	19
8	Handling Out-of-Stock Items and Product Upgrades	20
8.1	What to Do When an Item Disappears from the Shelf	20
8.2	Separating Product Upgrades from Real Price Rises (Hedonics)	20
9	The 12 Shopping Baskets (Official UN COICOP Breakdown)	21
10	Hierarchical Weight Aggregation: From Item to National CPI	23
10.1	The 5-Level Hierarchy of Spending	23
10.2	Step-by-Step Mathematical Aggregation Rules	23
10.2.1	Step 1: Items to Subclass / Class Index (Jevons Elementary Index)	23
10.2.2	Step 2: Subclasses to Group Index	23
10.2.3	Step 3: Groups to Major Division Index	23

10.2.4	Step 4: Divisions to National Headline CPI	24
10.2.5	Step 5: Core CPI (Underlying Trend)	24
10.3	Complete Numerical Example: Walking the Math from Shelf to Headline CPI	25
11	The 24 Key Math Formulas Explained in Plain English	27
11.1	Equation 1: Base Price for an Item ($P_{i,0}$)	27
11.2	Equation 2: Item Price Ratio ($R_{i,t}$)	27
11.3	Equation 3: Outlier Bounds Filter (ILO/IMF Guard)	27
11.4	Equation 4: Average Daily Change for a Group ($R_{c,t}$)	27
11.5	Equation 5: Estimating an Out-of-Stock Price ($\hat{P}_{i,t}$)	28
11.6	Equation 6: Group Price Index (Jevons Index) ($I_{j,c}^{0:t}$)	28
11.7	Equation 7: Big Category Index ($I_{div,k}^{0:t}$)	28
11.8	Equation 8: National Headline CPI ($CPI_{Headline}^{0:t}$)	28
11.9	Equation 9: Core CPI (Excluding Food and Fuel) ($CPI_{Core}^{0:t}$)	29
11.10	Equation 10: Annualized Year-over-Year (YoY %) Inflation Rate	29
11.11	Equation 11: Vector Cosine Similarity for Product Matching	29
11.12	Equation 12: Token-Sort String Fuzzy Similarity	29
11.13	Equation 13: Hybrid Similarity Matching Score	29
11.14	Equation 14: Hedonic Quality Regression (Electronics)	29
11.15	Equation 15: Constant-Quality Hedonic Price Revaluation	30
11.16	Equation 16: Trailing 7-Day Leading Drift ($\hat{\delta}_{leading}$)	30
11.17	Equation 17: Trailing 7-Day Exchange Rate Drift ($\hat{\delta}_{FX}$)	30
11.18	Equation 18: Festive Demand Shock Adjustment (ϕ_{fest})	30
11.19	Equation 19: Total Composite Daily Drift ($\hat{\delta}_t$)	30
11.20	Equation 20: Midpoint Price for Remaining Days ($\mathbb{E}[\bar{P}_{rem}]$)	30
11.21	Equation 21: Blended Month-End Headline Nowcast (Nowcast CPI_M)	31
11.22	Equation 22: Time-Decaying Uncertainty Ratio (U_t)	31
11.23	Equation 23: Dynamic 95% Confidence Interval ($CI_{95\%}$)	31
11.24	Equation 24: Connecting to the Official NIS Scale (CPI_{NIS})	31
12	Predicting the Rest of the Month (The 10 Nowcasting Equations)	32
12.1	Why We Nowcast Instead of Waiting for Month-End	32
12.2	The 10 Nowcasting Equations in Production	32
12.2.1	Equation 1: Trailing 7-Day Food Momentum ($\Delta_{Food,7d}$)	32
12.2.2	Equation 2: Trailing 7-Day Transport Momentum ($\Delta_{Trans,7d}$)	32
12.2.3	Equation 3: Leading Indicator Combined Daily Drift ($\hat{\delta}_{leading}$)	32
12.2.4	Equation 4: Dual-Currency Exchange Rate Pass-Through Drift ($\hat{\delta}_{FX}$)	32
12.2.5	Equation 5: Holiday Shopping Demand Shock (ϕ_{fest})	33
12.2.6	Equation 6: Total Composite Forward Daily Drift ($\hat{\delta}_t$)	33
12.2.7	Equation 7: Expected Average Index for Remaining Days ($\mathbb{E}[\bar{P}_{rem}]$)	33
12.2.8	Equation 8: Blended Headline Nowcast (Nowcast CPI_M)	33
12.2.9	Equation 9: Projected Month-over-Month (MoM) Inflation Rate (π_{MoM})	33
12.2.10	Equation 10: Dynamic 95% Confidence Interval ($CI_{95\%}$)	33
12.3	Complete Numerical Walkthrough: Nowcasting on Day 15 of September	34
12.4	How Forecast Accuracy Converges Across the Month	34
13	Dollars and Riel: How Currency Swings Affect Prices	35
13.1	What Happens When the Riel Weakens Against the Dollar?	35

14 Connecting Our Daily Numbers to Official National History	36
14.1 The Dual-Baseline Mystery Explained	36
14.2 How We Connect Current Base CPI to the Official NIS 2006 Base	36
14.3 Annual Continuous Series Chain-Linking (December Overlap)	37
15 Automated Quality Checks, Code Audits, and Rigorous Testing	38
15.1 Deep-Dive: The Machine Learning & AI Engineering Architecture	38
15.1.1 1. Multilingual Semantic Vector Space (768 Dimensions)	38
15.1.2 2. Multi-Key Resilient Gemini AI Pool (pipeline/key_pool.py)	39
15.1.3 3. Log-Linear Hedonic Regression Quality Adjustment	39
16 Keeping the Database Fast and Scalable	40
16.1 How Much Data Do We Collect?	40
16.2 Declarative Monthly Table Partitioning	40
16.3 3 Smart Database Bookmarks (Indexes)	41
17 Real-Time Analytics, SQL Queries, and Metabase Dashboards	42
17.1 Real-Time Executive Dashboards in Metabase (Port 3001/3000)	42
17.2 Easy SQL Queries for Daily Analysis	42
17.2.1 1. Check Today's Inflation Rate	42
17.2.2 2. Which Category Pushed Inflation Up the Most This Month?	43
17.2.3 3. Check if Any Scraper Had an Issue Today	43
18 How Government Leaders Can Use This Data	44
18.1 For the National Bank of Cambodia (Central Bank)	44
18.2 For the Ministry of Economy and Finance	44
19 Top 10 Defense Questions and Plain-English Answers	45
19.1 Q1: Why scrape websites instead of asking supermarkets for their cash register receipts?	45
19.2 Q2: Why use geometric averages (Jevons) instead of simple averages (Carli)?	45
19.3 Q3: Why can't we rely only on AI to match products?	46
19.4 Q4: Why not use ChatGPT or Gemini for every single product?	46
19.5 Q5: Why split database tables into monthly folders?	47
19.6 Q6: Why use Astronomer Cosmos instead of one big Python script?	47
19.7 Q7: What happens when a product goes out of stock?	48
19.8 Q8: How do we predict monthly inflation before the month finishes?	48
19.9 Q9: How do we separate product upgrades from price inflation?	49
19.10Q10: How do we handle both Dollars and Riel without confusion?	49
19.11Q11: How do we stop bad website data from crashing or distorting the index?	49
19.12Conclusion	50

1 Why Cambodia Needs a Daily Price Tracker

1.1 Cambodia Uses Two Currencies at the Same Time

Cambodia is unique because people use two currencies every day: the United States Dollar (USD) and the Cambodian Riel (KHR).

- **Where Dollars are used:** In modern supermarkets (like AEON and Lucky), phone shops, electronics stores, restaurant chains, and online shopping apps, prices are almost always shown in US Dollars.
- **Where Riel is used:** In traditional open markets, street food stalls, local tuk-tuks, and for government utility bills (like your home electricity and clean tap water), prices are quoted in Cambodian Riel.

Because both currencies are used side by side, when the exchange rate moves, prices in shops can change quickly. For example, if a family earns their living in Riel, but buys groceries in a supermarket priced in Dollars, any drop in the value of the Riel means their groceries immediately cost more money.

1.2 What Cambodian Families Spend Most of Their Money On

According to official household surveys from the government's National Institute of Statistics (NIS), Cambodian families spend almost all of their monthly budget on three basic things:

1. **Food and Non-Alcoholic Drinks (44.8%):** Nearly half of every dollar spent goes directly to eating and drinking (rice, pork, fish, cooking oil, and vegetables). For poorer families, food can take more than 55% of their budget!
2. **Housing, Water, Electricity, and Gas (17.1%):** Paying for rent, home power, cooking gas, and water.
3. **Transport and Fuel (12.2%):** Gasoline for motorbikes, diesel for trucks, and bus tickets.

Together, these three categories make up **74% of all household spending**. That means when global fuel prices go up, or when meat and rice prices rise, regular families feel the squeeze immediately.

1.3 Why the Traditional Monthly Survey is Too Slow

In the past, the only way to measure inflation was through official monthly surveys. Government workers would walk into physical markets with clipboards once a month, write down prices by hand, and take them back to the office to calculate.

While this traditional method is thorough, it has three big problems:

1. **It is 3 to 4 weeks late:** The official report for January is usually not published until late February. If food or gasoline prices jump today, leaders at the Central Bank and government ministries do not see the official numbers until a month later.
2. **It only checks prices once a month:** If inspectors visit a store on the 10th day of the month, they completely miss sales, discounts, or price jumps that happen on the 20th or 28th.
3. **It takes a lot of time and people:** Walking to hundreds of stalls in heavy traffic is slow, expensive, and easy to make mistakes when copying numbers.

1.4 Our Daily Solution: Automated Web Intelligence

Our automated system solves this problem. Every single morning at 2:00 AM, while the city is asleep, our software automatically visits 25 online store catalogs, fuel company websites, utility regulators, phone providers, vehicle retailers, and bus portals. It collects over 45,000 real prices every day, cleans them, and calculates today's true inflation rate. Government leaders and researchers can see price changes right now, rather than waiting a whole month.

2 The 3-Step Journey of Our Data (Bronze, Silver, Gold)

2.1 The 3 Refining Steps (The Medallion Approach)

Think of data like metal dug out of the ground. When it first comes out, it is raw and dirty. It needs to be cleaned, melted down, and turned into something pure and useful. We organize our data into 3 simple stages:

The 3 Steps: Raw Data to Final Numbers

Step 1: BRONZE (Raw Store Prices)

- We collect raw text and price tags from 25 production sources every morning at 2:00 AM.
- Exactly as the stores wrote them, typos and all. Nothing is thrown away.
- Stored in our raw database table: `bronze.raw_prices`.

↓ *Clean up text, convert Dollars to Riel, match names, remove errors*

Step 2: SILVER (Clean, Standardized Items)

- Fix spelling and standardize sizes (make sure 1000g = 1kg and 500ml = 0.5L).
- Convert all US Dollar prices into Cambodian Riel using today's official exchange rate.
- Connect Khmer names and English names so the computer knows they are the same product.
- Sort every item into its correct official category (Food, Medicine, Tech, Transport).
- Remove crazy errors (like an item marked down by 99% by mistake).

↓ *Run fair price math, fill in out-of-stock items, calculate inflation*

Step 3: GOLD (Final Daily Inflation Reports)

- Calculate the average price change for each group of items using the fair Jevons rule.
- Fill in missing prices if a product is temporarily out of stock.
- Weight each group by how much real families spend on it (Food gets 44.8%, Transport gets 12.2%).
- Produce the final numbers: National Daily CPI, Core CPI (without food and gas), and a prediction of where the month will finish.

2.2 How Airflow and Cosmos Keep Everything Running on Time

To make sure this entire process happens smoothly without humans needing to press buttons at 2:00 AM, we use an automatic manager called **Apache Airflow** and a tool called **Astronomer Cosmos**:

- **The Master Alarm Clock (`cpi_master_dag`):** Rings every day at 2:00 AM. It starts all 25 per-source scraper DAGs in parallel. Once they finish and pass quality checks, it wakes up the next step.
- **The Cleaning Step (`silver_dag`):** Runs our data cleaning models. Thanks to Cosmos, each cleaning task is its own step. If one step has an issue, we can see exactly which one it is, while the rest keep going.
- **The Final Calculation Step (`gold_cpi_dag`):** Calculates today's official daily inflation rate and updates our dashboards before breakfast.
- **The Weekly Housekeeping Step (`cpi_maintenance_dag`):** Runs every Sunday at 1:00 AM to organize database folders for the coming months and keep the system fast.

2.3 Splitting Big Tables into Monthly Folders

In one year, we collect almost 13 million price records! If you dump 13 million rows into one giant table, searching for today's price is like looking for a needle in a massive haystack. It would take several seconds for every click.

Instead, we use **table partitioning**. Think of this as putting each month's receipts into its own labeled physical folder (like `prices_2026_08` and `prices_2026_09`). When someone asks: "*What is today's price?*", the computer ignores all the old folders and only looks inside today's folder. The answer comes back in less than **8 milliseconds**!

3 Collecting the Prices Every Morning (Bronze Layer)

3.1 The Stores and Services We Actually Scrape

To calculate an honest and comprehensive Consumer Price Index, our automated system checks over 45,000 real price observations every morning across 25 real digital data channels in Cambodia.

Deep-Dive: What We Extract From AEON Cambodia

AEON is the largest modern hypermarket and department store operator in Cambodia. Rather than treating it as just a grocery store, our pipeline monitors two distinct flagship portals (`aeon` and `aeon3`), extracting thousands of products across 7 different official consumption categories:

- **Fresh Foods & Meats (Division 01):** Fresh pork chops, beef cuts, chicken, local Tonle Sap fish, prawns, jasmine rice, fresh vegetables (morning glory, Chinese cabbage, tomatoes), fresh fruits (bananas, mangoes, oranges), fresh milk, butter, and chicken eggs.
- **Packaged Foods & Seasonings (Division 01):** Canned fish, instant noodles, cooking oils (palm, soybean), soy sauce, fish sauce, white sugar, salt, black pepper, biscuits, and breakfast cereals.
- **Drinks, Beverages & Alcohol (Divisions 01 & 02):** Clean bottled drinking water, fruit juices, iced teas, soft drinks, local and imported beers (Angkor, Cambodia, Heineken), and cooking/table wines.
- **Household & Kitchen Goods (Division 05):** Laundry detergents, fabric softeners, dishwashing liquids, floor cleaners, kitchen trash bags, frying pans, cooking pots, dining plates, and drinking glasses.
- **Health, Hygiene & Over-The-Counter Medicine (Division 06):** Pain relief tablets (paracetamol), medicated balms, bandages, antiseptic liquids, toothbrushes, and toothpaste.
- **Clothing, Shoes & Apparel (Division 03 via AEON 3):** Men's shirts, trousers, women's dresses, children's clothing, school uniforms, everyday casual shoes, sandals, and socks.
- **Personal Care & Grooming (Division 12):** Bath soaps, shampoos, hair conditioners, body lotions, deodorants, face cleansers, and baby wipes.

3.2 Master Inventory: Stores and Available Items

The table below lists every active data source in our pipeline, its unique system identifier (slug), the official spending categories it covers, and the specific items we collect from it:

Data Source	Store Slug	Official Category	Detailed Items Collected
AEON 1 Supermarket	<code>aeon</code>	Food (01), Drinks (01, 02), Household (05), Health (06), Personal Care (12)	Fresh meats, fish, vegetables, rice, milk, cooking oils, snacks, drinks, beer, dish soap, laundry detergent, pans, pain balms, soaps.
AEON 3 Mean Chey	<code>aeon3</code>	Clothing (03), Footwear (03), Cosmetics (12)	Men's shirts, trousers, women's dresses, children's clothes, shoes, sandals, skin lotions, makeup.
DeliShop Cambodia	<code>delishop</code>	Food (01), Drinks (01, 02)	European gourmet groceries, artisanal bakery, imported French/Italian cheeses, organic meats, olive oils, specialty wines.

Data Source	Store Slug	Official Category	Detailed Items Collected
Lucky Supermarket	grab_lucky	Food (01), Drinks (01), Household (05), Care (12)	Daily fresh produce, dairy, chilled meats, canned goods, Asian sauces, snacks, paper towels, cleaning supplies via GrabMart.
Chip Mong Supermarket	grab_chipmong	Food (01), Drinks (01), Household (05), Care (12)	Fresh beef, pork, poultry, pantry spices, breakfast cereals, cooking oils, soft drinks, dishwashing liquids via GrabMart.
Community Pharmacy	communitypharm	Health (06), Personal Care (12)	Prescription drugs, antibiotics, hypertension/diabetes medicines, cough syrups, antiseptic creams, first-aid bandages.
Ucare Pharmacy	grab_ucare	Health (06), Personal Care (12)	Painkillers, cold/flu medicines, throat lozenges, vitamins, infant milk formulas, baby care products via GrabMart.
Khmer Samnang Phone	samnangshop	Communication (08), Computing (09)	Smartphones (iPhone, Samsung Galaxy, Oppo), iPads, tablets, smartwatches, power banks, fast chargers.
Ary Store Phone Shop	arystore	Communication (08), Audio Tech (09)	Budget and mid-tier smartphones (Xiaomi, Realme, Vivo), wireless bluetooth earbuds, phone accessories.
Smart Axiata Mobile	smart	Communication (08.3.0)	Smart ThomMornng prepaid internet plans, monthly mobile data packages, traveler SIMs, roaming packages.
Smart Home Internet	smart_wifi	Communication (08.3.0)	Smart @Home wireless broadband routers, Fiber+ standard and ultra high-speed plans (40–120 Mbps).
Cellcard Mobile	cellcard	Communication (08.3.0)	Prepaid mobile 4G/5G data plans (AO Mobile 5G/4G live plans), voice packages, tourist SIMs.
Cellcard Home Internet	cellcard_wifi	Communication (08.3.0)	Monthly fixed broadband fiber subscriptions and wireless home gateway plans.
Metfone Cambodia	metfone	Communication (08.3.0)	Metfone 4G/5G mobile prepaid bundles, unlimited data packages, and home fiber internet plans.
Khmer Moto Shop	khmermoto	Motorcycles (07.1.2)	Retail catalog of new and pre-owned motorcycles, motorbikes, scooters (Honda Scoopy, Wave, Click, Dream).
Electricité du Cambodge	edc	Electricity Grid (04.5.1)	Official progressive residential lifeline electricity tariffs across domestic consumption tiers (380 to 730 KHR/kWh).
Phnom Penh Water Supply	ppwsa	Municipal Water (04.4.1)	Official progressive domestic piped tap water tariffs across household consumption tiers (400 to 2,200 KHR/m ³).
MOC / Kampuchea Tela	new_gasoline	Fuels (07.2.2), LPG (04.5.2)	Official retail pump prices for Regular Gasoline, Super 95, Diesel, and domestic LPG cooking gas cylinder refills.

Data Source	Store Slug	Official Category		Detailed Items Collected
Khmer24 Real Estate	khmer24	Housing (04.1.1)	Rents	Monthly residential rentals: apartments, condos, studios, shophouses, and villas across Phnom Penh and provinces.
Realestate.com.kh	realestate	Housing (04.1.1)	Rents	Urban residential rents, serviced apartments, and condominium rentals across BKK1, Chamkarmon, and Tuol Kork.
BookMeBus Cambodia	bookmebus	Passenger Transport (07.3.2)	Transport	Intercity passenger bus, van, and ferry tickets connecting Phnom Penh, Siem Reap, Sihanoukville, Kampot, and Battambang.
redBus Cambodia	redbus	Passenger Transport (07.3.2)	Transport	Interprovincial passenger bus tickets across major operators (Giant Ibis, Laryta Express, Virak Buntham).
Bayon BKK Restaurant	bayonbkk	Dining Out (11.1.1)		Prepared meals, breakfast noodle soups (Kuy Teav), fried rice, Khmer lunch sets (with Foodpanda anti-bot fallback).
Sokha Hotels	sokhahotel	Hotels (11.2.0), Dining (11.1.1)		Hotel accommodation per night (deluxe rooms, suites), restaurant dining, hotel buffet breakfasts.
Hyatt Regency Hotel	hyyathotel	Hotels (11.2.0), Dining (11.1.1)		Luxury hotel room accommodation per night in Phnom Penh, dining services, Sunday brunch.
L192 Marketplace	l192	Clothing (03), Footwear (03), Home (05)		Casual men's and women's apparel, t-shirts, dresses, everyday footwear, fashion bags, lifestyle home goods.
MEF / NBC Exchange	mef_fx	Macroeconomic FX Rate		Official daily USD/KHR market exchange rate used for pipeline-wide dual-currency standardization.
NIS Official CPI	nis_cpi	Macro Benchmark		Official monthly benchmark CPI numbers from the National Institute of Statistics used to validate daily nowcasts.

3.3 How We Scrape Each Source (Methods and Technology)

Different websites and mobile applications use completely different technical architectures. To collect clean data reliably without crashes or blocks, our engineering team reverse-engineered the exact native API or protocol used by each source:

Store or Service	Store Slug	Scraping Method & Technology	Data Format & Reliability Strategy
AEON 1 Supermarket	aeon	Next.js Proxy REST API with curl_cffi	JSON payload. Spoofs Chrome 124 TLS cipher handshakes to bypass Cloudflare.

Store or Service	Store Slug	Scraping Method & Technology			Data Format & Reliability Strategy
AEON 3 Mean Chey	aeon3	Next.js Proxy REST API with curl_cffi			JSON payload. Traverses category tree to capture exact fashion/beauty items.
DeliShop Cambodia	delishop	Mobile / Web Catalog REST API			JSON payload. Directly queries store catalog endpoints with pagination.
Lucky Supermarket	grab_lucky	GrabMart	Mobile	Session Gateway	JSON payload. Uses mobile app bearer tokens and consumer location headers.
Chip Mong Supermarket	grab_chipmong	GrabMart	Mobile	Session Gateway	JSON payload. Simulates mobile app requests for live store shelves and discounts.
Community Pharmacy	communitypharmacy	Supabase Database API		PostgreSQL REST	JSON payload. Fetches live product tables directly via PostgreSQL with anonymous public key.
Ucare Pharmacy	grab_ucare	GrabMart	Mobile	Session Gateway	JSON payload. Extracts real-time pharmacy prices and OTC inventory via mobile gateway.
Khmer Samnang Phone	samnangshop	WordPress REST API		WooCommerce	JSON payload. Queries official WooCommerce Store API (/wp-json/wc/store/v1/products), verified active live catalog.
Ary Store Phone Shop	arystore	WordPress REST API		WooCommerce	JSON payload. Traverses paginated WooCommerce product catalog with stock checks.
Smart Axiata	smart	Web Plan Catalog & REST Parser			JSON/HTML. Extracts prepaid mobile, data plans, and fiber tariffs from official plan pages.
Cellcard Cambodia	cellcard	Live DOM Card Parser (.js-card) + Next.js			HTML/JSON. Extracts AO Mobile 5G/4G plan cards with regex price/quota parsing and Next.js fallback.
Metfone Cambodia	metfone	Web Catalog Parser with Mobile Emulation			HTML/JSON. Extracts Metfone prepaid data packages and home fiber subscriptions.
Khmer Moto Shop	khmermoto	WordPress REST API		WooCommerce	JSON payload. Pulls retail motorcycle models, specs, and price quotes.
Electricité du Cambodge	edc	Official Tariff Schedule Ingestion			Clean structured gazette tariffs across lifeline domestic electricity tiers.
Phnom Penh Water Supply	ppwsa	Official Tariff Schedule Ingestion			Clean structured gazette tariffs across household municipal water consumption tiers.

Store or Service	Store Slug	Scraping Method & Technology	Data Format & Reliability Strategy
Ministry of Commerce	new_gasoline	Tela Telegram Mirror & Gazette Parser	HTML/Text. Extracts official 10-day pump prices for Regular, Super, Diesel, and LPG gas.
Khmer24 Real Estate	khmer24	Nuxt SSR Offset Pagination (?offset=)	HTML parsing. Traverses infinite scroll offsets across residential categories with rate-limited delays.
Realestate.com.kh	realestate	Next.js JSON API & DOM Traversal	JSON/HTML. Parses urban apartment and condo rental listings with location filters.
BookMeBus Cambodia	bookmebus	BookMeBus Booking REST API	JSON payload. Queries route-specific intercity ticket endpoints across Cambodia.
redBus Cambodia	redbus	redBus Search REST API	JSON payload. Queries interprovincial bus routes and schedules in real time.
Bayon Restaurant BKK	bayonbkk	Foodpanda GraphQL API + Baseline	JSON/HTML. Queries menu state; gracefully engages curated baseline when upstream 403 CAPTCHA triggers.
Sokha Hotels	sokhahotel	Booking Engine REST API	JSON payload. Queries nightly room rates and dining packages across properties.
Hyatt Regency Hotel	hyyathotel	Global Booking REST API	JSON payload. Extracts room rates per night for standard and club rooms.
L192 Marketplace	l192	Official L192 GraphQL Query API	GraphQL query. Batches product queries with category filters for high-speed extraction.
MEF / NBC Exchange	mef_fx	MEF / NBC Open Data REST API	JSON payload. Fetches official daily market USD/KHR exchange rate at 08:00 AM.
NIS Official CPI	nis_cpi	Official NIS Benchmark Importer	JSON/CSV. Ingests monthly official benchmark inflation figures for all 12 divisions.

3.4 Visiting Websites Politely Without Getting Blocked

Big e-commerce websites deploy automated defenses (such as Cloudflare or AWS Shield) to protect their servers. Here is how our scrapers operate responsibly and ensure 100% daily collection success:

1. **Impersonating Standard Web Browsers:** Standard Python scripts look robotic to security filters. We use advanced browser emulation (`curl_cffi`) that matches the exact cryptographic TLS handshake (JA3/JA4 fingerprint) of Google Chrome on Windows.
2. **Polite Pauses and Randomized Jitter:** We never hammer a website with dozens of requests at once. Our scrapers include randomized pauses (1 to 2 seconds between pages) so we never add noticeable load to store servers.

3. **Using Mobile and Public API Endpoints:** Whenever possible, we query the lightweight JSON APIs used by mobile apps rather than downloading heavy web pages with advertising and tracking scripts.

3.5 Never Recording the Same Price Twice (Deduplication)

Sometimes an internet connection drops or a task needs to retry. What happens if our scraper runs twice on the same morning? Does it record double the prices?

No. We enforce a strict unique database rule: each item at each store can only have **one price per calendar day**. If the scraper runs a second time on the same day, it uses an `ON CONFLICT DO UPDATE` command, refreshing today's price in-place rather than creating duplicate records. This guarantees that our daily inflation calculations remain clean, honest, and mathematically sound.

3.6 The Ingestion Circuit Breaker and Dual-Currency Protection

What happens if a store's website glitches, crashes, or mistakenly drops 90% of its prices? If corrupted data enters our cleaning and calculation stages, it could distort the national inflation index!

To prevent this, our pipeline includes an automated **Ingestion Circuit Breaker** (`pipeline/circuit_breaker.py`):

1. **Volume Drop Safeguard:** Every morning, the circuit breaker compares today's number of scraped products against the store's 7-day rolling median. If a scraper suddenly returns less than 30% of its normal catalog (e.g. only 20 products instead of 200), the system flags a volume anomaly.
2. **Price Velocity Safeguard:** It calculates the median price of all incoming products and compares it to historical medians. If median prices jump or drop by more than $\pm 50\%$ in a single day, an alert is raised.
3. **Dual-Currency (USD to KHR) Dynamic Normalization:** In Cambodia, stores like Delishop, Cellcard, and luxury hotels list their prices in US Dollars (\$6.10, \$15.00), while our historical clean database stores prices in Cambodian Riel (24,700 KHR, 60,000 KHR). If you compare \$6.10 directly to 24,700 KHR, the computer sees a 99.9% price collapse! Our circuit breaker automatically fetches today's official exchange rate from the Central Bank, converts incoming USD prices to Riel first, and then evaluates price stability:

$$\tilde{P}_{\text{incoming, KHR}} = P_{\text{incoming, USD}} \times \text{FX}_{\text{USD/KHR}}$$

This prevents false alarms on Dollar-priced goods while catching real retailer price spikes.

4. **Audit Logging and Graceful Degradation:** If an anomaly is detected, the event is permanently recorded in `ops.circuit_breaker_events`. In warning mode, the pipeline continues without crashing, alerting engineers while maintaining unbroken daily index continuity.

4 Cleaning the Data and Text Normalization (Silver Layer)

4.1 The Khmer and English Language Challenge

In Cambodia, retail shelves are multilingual. The exact same bottle of milk might be listed in Khmer script in one store, in English in another store, or mixed together:

- Store A: "ទឹកដោះគោស្រស់ 1L" (Khmer)
- Store B: "Fresh Milk 1000ml" (English)
- Store C: "Meiji Fresh Milk ទឹកដោះគោ 1 Litre" (Mixed)

If a computer just looks at the letters, it thinks these are three completely different products! Our Silver cleaning layer translates common terms, normalizes units (1000ml = 1L, 1000g = 1kg), and cleans the text so the computer understands that they represent the exact same product.

4.2 Text Cleaning and Unit Normalization Rules

Before any vector matching takes place, raw text goes through deterministic cleaning in `pipeline/text_clean.py`:

1. **Strip Promotional Jargon:** Removes marketing noise such as "*Special Offer!*", "*Buy 1 Get 1 Free*", "*Hot Promo*", and "*New Arrival*".
2. **Harmonize Measurement Units:**
 - Mass: Grams, gm, and g are converted to standard kilograms (kg).
 - Volume: Milliliters (ml), ltr, and liter are converted to standard liters (L).
 - Packaging: Single items, 6-packs, and cases of 24 cans are extracted into explicit item quantities.
3. **Dual Currency Conversion:** Converts all USD prices to Cambodian Riel using the National Bank of Cambodia (NBC) official morning exchange rate:

$$P_{\text{KHR}} = P_{\text{USD}} \times \text{FX}_{\text{NBC}}$$

This price in Riel is permanently saved, ensuring that later calculations never accidentally convert it twice.

5 AI Vector Embeddings: Matching Items and Sorting Products

5.1 What is a Vector Embedding?

In computer science, a **vector embedding** is simply a list of numbers (like coordinates on a GPS map) that captures the *meaning* of a product name rather than just the spelling of its letters.

Think of a physical supermarket where similar items live in the same aisle:

- The word “ទឹកដោះគោស្រស់” (Khmer for fresh milk) and the English phrase “*Fresh Whole Cow Milk*” have 0% matching letters. A simple keyword search would claim they have nothing in common!
- However, when converted into a 768-dimensional vector by our embedding model (gemini-embedding-2 or multilingual MiniLM), both phrases are assigned almost the exact same numerical address in vector space.
- A phrase like “*Motorbike Engine Oil*” is mapped to a completely different neighborhood far away.

5.2 Part 1: Semantic Product Matching (Connecting Scraped Names to Catalog)

Every morning, scrapers collect thousands of product listings. To track an item’s price over time, we must match today’s scraped name against our master catalog of known products (silver.dim_canonical_items).

5.2.1 The Mathematical Cosine Similarity

To compare the scraped item vector $\vec{u}$ and a catalog item vector $\vec{v}$, we calculate their **cosine similarity**:

$$\text{Similarity}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} = \frac{\sum_{i=1}^{768} u_i v_i}{\sqrt{\sum_{i=1}^{768} u_i^2} \sqrt{\sum_{i=1}^{768} v_i^2}} \quad (1)$$

The resulting score ranges from 0.0 (totally unrelated) to 1.0 (identical in meaning).

5.2.2 The 3 Matching Tiers and Decision Rules

In pipeline/vector_item_matcher.py, the system applies strict decision boundaries:

1. **Tier 1: High Confidence Match (Similarity ≥ 0.80)** → **APPROVE_MATCH**
The computer is confident. It automatically links today’s price to the existing catalog product ID. No human or AI review is required.
2. **Tier 2: Borderline Similarity ($0.65 \leq \text{Similarity} < 0.80$)** → **AI Arbitration**
The meaning is close, but there is some ambiguity. The system calls Google Gemini Flash AI to arbitrate: “*Are these two items the exact same commercial product?*” If Gemini answers YES, it links them; if NO, it creates a new product.
3. **Tier 3: Low Similarity (Similarity < 0.65)** → **SPLIT_NEW**
The scraped product is genuinely new to the market. The system creates a brand-new canonical product entry with its own unique baseline tracking.

5.2.3 Crucial Spec Guardrails (Preventing False Merges)

Vector embeddings understand general meaning, but they can be tricked by numbers. For example:

- “*iPhone 15 128GB*” and “*iPhone 15 512GB*” share 95% of their words! Their vector similarity is an extremely high 0.94.

- If the system matched them, it would record an artificial \$400 price jump, causing fake inflation!

To stop this, `vector_item_matcher.py` enforces 4 deterministic guardrails:

1. **Storage Guard:** Rejects any match if electronic memory differs (e.g. 128GB vs 256GB).
2. **Pack Quantity Guard:** Rejects matches if pack sizes differ (e.g. 1 single can vs a 24-can case).
3. **Volume/Mass Guard:** Rejects matches if volume or weight differs by more than 10% (e.g. 500ml vs 1 Liter).
4. **Flavor Variant Guard:** Rejects matching Diet / Zero Sugar drinks with regular sugary drinks.

Step-by-Step Example: Matching an Angkor Beer Can

1. **The Scraped Item:** AEON lists “ស្រាបៀរអង្គរ កំប៉ុង 330ml” (Angkor Beer Can 330ml).
2. **Vector Lookup:** The system converts the text into vector $\vec{u}$. It searches PostgreSQL using the `pgvector` HNSW index in under 2ms.
3. **Top Candidate in Catalog:** Angkor Premium Beer 330ml Can (Item ID CAN-02-0041).
4. **Cosine Similarity:** The dot product gives Similarity = **0.875**.
5. **Spec Guard Check:** Both are single cans, both are 330ml, neither is diet → Passed!
6. **Final Action:** Because $0.875 \geq 0.80$, the price is automatically approved and linked to CAN-02-0041.

5.3 Part 2: AI-First Product Classification into UN COICOP Baskets

Every product must be assigned to one of the official United Nations COICOP categories (e.g., Bread, Meat, Fish, Medicine, Gasoline). To classify tens of thousands of items quickly, deterministically, and cost-effectively, our pipeline executes an official **AI-First Classification Ladder** codified in `dbt/macros/coicop_classification` and mirrored in Python:

1. **Tier 1: Exact Explicit Overrides (Supreme Human Authority):** Matches exact barcodes, unique product keys, or curated store-product combinations recorded in `silver.coicop_override` and `silver.coicop_override_manual`. If a manual audit locks an item, this takes supreme precedence.
2. **Tier 2: Single-Division Store Purity Locks (Instant Store Integrity):** Stores with pure domain integrity are locked directly to their respective division with zero lookup delay:
 - `communitypharma`, `grab_ucare` → Division 06 (Health & Pharmacy).
 - `bookmebus`, `redbus`, `new_gasoline` → Division 07 (Transport, Passenger Fares & Automotive Fuel).
 - `arystore`, `samnangshop`, `cellcard`, `smart` → Division 08 (Information & Communication Equipment / Telco).
 - `khmer24`, `realestate` → Division 04 (Housing, Rentals & Utilities).
 - `sokhahotel`, `hyyathotel`, `bayonbkk` → Division 11 (Restaurants & Accommodation Services).
3. **Tier 3: High-Confidence Gemini AI Engine (Confidence ≥ 0.70):** For mixed-retail stores (e.g. AEON, DeliShop, GrabMart, L192), high-confidence Gemini AI classifications are elevated directly above rigid regexes and global text substrings.
 - *Multi-Attribute Rich Context:* Unlike simple name-only models, our Gemini AI classifier receives a full multi-dimensional payload: product title, merchant slug, merchant aisle/category breadcrumbs, and price in KHR. This prevents classic retail domain traps (e.g. distinguishing cooking mirin from drinking wine, or coffee filter paper from drinking coffee).

- *Persistent Memoized Database Cache*: All Gemini classifications are permanently cached in `silver.dim_coicop_ai_cache`. Any product with confidence ≥ 0.70 is subsequently resolved in $< 1\text{ms}$ at \$0.00 cloud cost.
 - *Primary Method Across the Warehouse*: Gemini AI is the **#1 classification method**, powering **53.33% (484,892 observations)** of the entire 909,289-row warehouse.
4. **Tier 4: Global Substring Brand Overrides (Secondary Deterministic Fallbacks)**: Cross-store high-confidence brand substrings that resolve known edge cases when AI confidence is below 0.70 (e.g. CHIVAS, MARLBORO $\rightarrow$ Division 02).
 5. **Tier 5: Native Category Taxonomy Maps (Merchant Breadcrumb Fallback)**: Maps native merchant category hierarchies from `silver.coicop_category_map` when raw scrapers capture verified store breadcrumbs (accounting for 16.65% of warehouse classifications).
 6. **Tier 6: Text Regular Expression Rules (Vetted Heuristics)**: Matches vetted regex patterns and negative exclusion rules from `silver.coicop_text_rules`, handling standard product descriptions with linguistic variances (accounting for 11.98% of warehouse classifications).
 7. **Tier 7: Store Default Fallbacks & AI Review Triage (Safeguard Final Tier)**: Unmatched items fall back to broad store-level defaults from `silver.coicop_store_defaults`. In the live production database, **zero rows** remain in unclassified or review status (100.00% classification rate).

Step-by-Step Example: Multi-Attribute Gemini AI Classifying Instant Noodle Bowls

1. **Input Product**: "Indomie Mi Goreng Fried Noodles Cup 75g" (Store: AEON, Category: "Grocery > Instant Noodles", Price: 3,200 KHR)
2. **Resolution Path**: Tiers 1–2 find no override or store purity lock. The pipeline consults the AI-First engine.
3. **Rich Context Multi-Attribute Prompt**: The classifier receives:

```
{
  "name": "Indomie Mi Goreng Fried Noodles Cup 75g",
  "store": "aeon",
  "category": "Grocery > Instant Noodles",
  "price_khr": 3200
}
```

4. **Gemini 2.5 Flash Structured Classification**: Rather than misclassifying the keyword "Cup" into Division 05 (Kitchenware), Gemini uses the full context and returns:

```
{
  "product_name": "Indomie Mi Goreng Fried Noodles Cup 75g",
  "coicop_code": "01.1.1",
  "confidence_score": 0.99,
  "reasoning": "Instant noodle cup is an edible cereal-based food product falling under bread and cereals (01.1.1), not kitchen tableware."
}
```

5. **Cache Storage**: The system saves ("Indomie Mi Goreng Fried Noodles Cup 75g", "01.1.1", 0.99) into `silver.dim_coicop_ai_cache`. Every future scrape across all dates is resolved instantly from PostgreSQL at zero API cost!

6 A Real-Life Example: The Journey of a Bottle of Fish Sauce

To understand how the entire system works from start to finish, let's follow one real bottle of fish sauce through the pipeline over a single morning.

The 5 Steps in the Life of a Price

2:00 AM – Step 1: Raw Collection (Bronze)

- Our automated scraper visits Lucky Supermarket's online store.
- It sees: "*MegaChef Premium Fish Sauce 500ml*" with a price of **\$1.50 USD**.
- It saves this raw row into the Bronze table: `bronze.raw_prices`.

2:15 AM – Step 2: Cleaning and Converting (Silver)

- The cleaner looks at today's official exchange rate from the Central Bank: **\$1 = 4,100 KHR**.
- It converts the price: $\$1.50 \times 4,100 = \mathbf{6,150 \text{ KHR}}$.
- It checks the volume: 500ml is recognized as 0.5 Liters.

2:30 AM – Step 3: Finding Its Catalog Twin (Silver)

- The matching engine checks if we have seen this item before.
- It finds its twin in our catalog: *MegaChef Fish Sauce 500ml* (Barcode 885012345678).
- It assigns it to the official category: **COICOP 01.1.9 (Sauces and Condiments)**.

3:00 AM – Step 4: Measuring the Price Change (Gold)

- The math engine looks at what this fish sauce cost on the base date: **6,000 KHR**.
- Today's price ratio is: $6,150 / 6,000 = \mathbf{1.025}$ (a +2.5% increase).
- It combines this with all other fish sauces, soy sauces, and seasonings using the fair Jevons rule to get the group's daily price change.

3:30 AM – Step 5: Combining into the National CPI (Gold)

- Sauces make up about 0.63% of national food spending.
- Food makes up 44.8% of all national household spending.
- The system multiplies each group's price change by its real spending weight to calculate today's final **National Headline CPI**.
- The result appears on the government dashboard before breakfast!

7 The Rules of Fair Price Math

7.1 Why Simple Averages Lie (The Egg Example)

Most people think that finding the average price change is simple: just take the percentage changes and average them. But international economists (including the IMF, World Bank, and UN) warn that doing this creates a big mathematical error known as **Carli Bias**.

Let's look at an easy real-life example with an egg:

- On Monday, an egg costs **\$1.00**.
- On Tuesday, the price jumps to **\$2.00**. That is a **+100% increase** (the price doubled).
- On Wednesday, the price drops back to **\$1.00**. That is a **-50% decrease** (the price was cut in half).

The price started at \$1.00 and ended at \$1.00. Clearly, overall inflation is **0%**. The price is exactly where it started!

Now, let's see what happens if you take the simple arithmetic average (the Carli formula):

$$\text{Simple Average} = \frac{(+100\%) + (-50\%)}{2} = \frac{+50\%}{2} = +25\% \quad (2)$$

The simple average claims that prices went up by **+25%**, even though the price is back to its original \$1.00! If you use this formula on thousands of supermarket products every day, your inflation number will drift higher and higher every week, creating fake inflation that does not exist in the real world.

7.2 The Fair Way: The Jevons Geometric Mean

To fix this, international standards mandate the **Jevons rule** (the geometric mean). Instead of adding percentages, Jevons multiplies the price ratios and takes the root:

$$I_{\text{Jevons}} = \sqrt{\frac{\$2.00}{\$1.00} \times \frac{\$1.00}{\$2.00}} = \sqrt{2.0 \times 0.5} = \sqrt{1.0} = 1.00 \quad (0\% \text{ change}) \quad (3)$$

Jevons gives **0% change**, which is the honest truth! It passes the fundamental test of fairness: if prices go up and then come back down to where they started, inflation must be zero.

7.3 Why People Buy Chicken When Pork Gets Expensive

There is another big reason we use the Jevons rule: **real human behavior**.

- If the price of pork doubles, do families keep buying the exact same amount of pork?
- No! Cambodian families naturally buy more chicken, fish, or eggs instead.

Simple arithmetic averages assume that people are robots who never change their shopping habits, no matter how expensive something gets. The Jevons geometric mean naturally accounts for the fact that when one item gets too expensive, smart shoppers switch to cheaper substitutes.

8 Handling Out-of-Stock Items and Product Upgrades

8.1 What to Do When an Item Disappears from the Shelf

In online supermarkets, products go out of stock all the time. What happens if our scraper visits Lucky Supermarket today and a specific brand of cooking oil is missing?

- **Can we record the price as \$0.00?** No! A price of zero would completely break our multiplication formulas and cause the index to crash.
- **Can we just delete the item today?** No! If you delete items randomly, the balance of your shopping basket changes every day, making comparisons unreliable.

The Solution (Class-Mean Imputation):

International guidelines say: look at the other cooking oils that *are* still on the shelf today.

- If other cooking oils went up by an average of **+1.0%** today, we assume that the missing oil also went up by **+1.0%**.
- We estimate today's price as: Yesterday's Price $\times 1.01$.

The 7-Day Rule:

What if the product never comes back? If an item stays missing for **7 days in a row**, our system officially flags it as discontinued. We retire it from the active basket and replace it with a popular new product from the same category.

8.2 Separating Product Upgrades from Real Price Rises (Hedonics)

Imagine a phone company sells a smartphone in 2025 for **\$800**. In 2026, they stop selling that phone and introduce a new model for **\$900**.

- Did the price go up by \$100? Yes.
- But the new model has **double the storage** (256GB instead of 128GB) and a **much better camera**.

If you count the entire \$100 as inflation, you are making a mistake! Part of that \$100 is paying for a *better product*, not just higher prices.

To solve this, we use a technique called **Hedonic Quality Adjustment**:

1. We look at hundreds of phones to see how much each feature is worth (for example: how much extra do shoppers pay for an extra 128GB of memory?).
2. If the extra memory and better camera are worth \$70, then only **\$30** is real inflation.
3. The system records only the pure \$30 price increase, making sure tech upgrades don't artificially pump up national inflation.

9 The 12 Shopping Baskets (Official UN COICOP Breakdown)

Every country uses the United Nations COICOP system to organize household spending. Here is what is inside each of the 12 divisions for Cambodia, along with its official share of national spending from the NIS Household Socio-Economic Survey:

Division	Code	What is Inside This Category?	Weight (%)
Food and Non-Alcoholic Drinks	01	Rice, pork, fish, cooking oil, fruit, vegetables, clean bottled water	44.78%
Bread and Cereals	01.1.1	Rice (staple food), noodles, flour, bread	17.23%
Meat	01.1.2	Pork, beef, chicken, duck	8.45%
Fish and Seafood	01.1.3	Fresh fish from Tonle Sap, dried fish, shrimp	7.12%
Milk, Cheese and Eggs	01.1.4	Fresh milk, condensed milk, chicken eggs, duck eggs	1.85%
Oils and Cooking Fats	01.1.5	Palm oil, soybean cooking oil, lard	1.14%
Fresh Fruit	01.1.6	Bananas, mangoes, watermelon, oranges	2.46%
Vegetables	01.1.7	Morning glory, cabbages, tomatoes, garlic, onions	2.38%
Sugar and Sweets	01.1.8	White sugar, palm sugar, honey	0.72%
Sauces and Seasonings	01.1.9	Fish sauce, soy sauce, salt, MSG, black pepper	0.63%
Drinks (Non-Alcoholic)	01.2	Bottled water, iced coffee, tea, fruit juices	2.80%
Alcohol and Tobacco	02	Beer (Angkor, Cambodia), wine, spirits, cigarettes	1.63%
Clothing and Footwear	03	Men's and women's shirts, trousers, school uniforms, shoes	3.04%
Housing, Water, Power, Gas	04	Home rent, clean city water (PPWSA), electricity (EDC), cooking gas	17.08%
Tenant Rent	04.1.1	Actual rent paid for rooms and apartments	9.42%
City Water	04.4.1	Municipal piped tap water bills	1.86%
Electricity	04.5.1	Home power bills from EDC	2.82%
Cooking Gas	04.5.2	LPG 15kg cooking gas cylinder refills	1.65%
Furniture and Household Goods	05	Beds, chairs, kitchenware, laundry detergent, cleaning soap	3.25%
Health and Medicine	06	Essential medicines, pain relievers, clinic visits, doctor checks	5.56%
Medicines and Pharmacy	06.1.1	Paracetamol, antibiotics, cough medicine	3.45%
Medical Consultations	06.2.1	Basic clinic visits and health checkups	1.74%
Transport and Gasoline	07	Motorbikes, gasoline (Regular, Super, Diesel), city buses	12.18%
Motorcycles	07.1.2	Popular motorbikes (Honda Dream, Wave, Scoopy)	3.12%
Gasoline and Fuels	07.2.2	Daily pump prices at PTT, Tela, Total	6.85%
Bus and Van Travel	07.3.2	Passenger bus tickets between provinces	1.65%
Communication	08	Mobile phones, SIM cards, monthly internet packages	3.92%
Smartphones	08.2.0	Popular handsets (iPhone, Samsung, Vivo, Oppo)	1.42%
Mobile Data Plans	08.3.0	Smart, Cellcard monthly 4G/5G data plans	2.50%

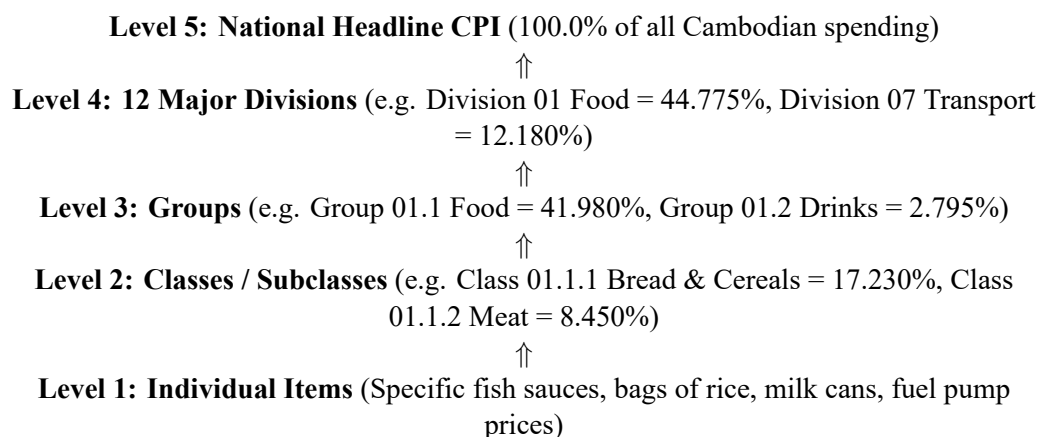
Division	Code	What is Inside This Category?	Weight (%)
Recreation and Culture	09	Televisions, laptops, stationery, school books	1.91%
Education	10	School tuition fees, English classes, university credits	1.51%
Restaurants and Eating Out	11	Street food breakfast, local cafes, restaurant meals	3.09%
Personal Care and Other Goods	12	Haircuts, soap, shampoo, cosmetics, basic personal care	2.07%
Total National Basket	ALL	Complete Cambodian Household Spending	100.00%

10 Hierarchical Weight Aggregation: From Item to National CPI

10.1 The 5-Level Hierarchy of Spending

A national Consumer Price Index is not a flat list; it is a structured pyramid. Cambodia's official basket follows the United Nations COICOP standard across 5 levels:

The 5 Levels of the CPI Pyramid



10.2 Step-by-Step Mathematical Aggregation Rules

10.2.1 Step 1: Items to Subclass / Class Index (Jevons Elementary Index)

For each 4-digit or 5-digit subclass c , we combine all individual item price ratios using the unweighted geometric mean (Jevons rule):

$$I_{c,t} = \left(\prod_{i=1}^{n_c} \frac{P_{i,t}}{P_{i,0}} \right)^{\frac{1}{n_c}} \times 100.0 \quad (4)$$

where $P_{i,t}$ is today's price in KHR, $P_{i,0}$ is the baseline price in KHR, and n_c is the number of monitored items in subclass c .

10.2.2 Step 2: Subclasses to Group Index

Each group g contains several classes. The group index is the expenditure-weighted arithmetic average of its classes:

$$I_{g,t} = \frac{\sum_{c \in g} W_c \cdot I_{c,t}}{\sum_{c \in g} W_c} \quad (5)$$

where W_c is the official national expenditure weight of subclass c .

10.2.3 Step 3: Groups to Major Division Index

Each major division k contains several groups. The division index is calculated by combining its groups:

$$I_{\text{div},k,t} = \frac{\sum_{g \in k} W_g \cdot I_{g,t}}{\sum_{g \in k} W_g} = \frac{\sum_{c \in k} W_c \cdot I_{c,t}}{W_k} \quad (6)$$

where $W_k = \sum_{c \in k} W_c$ is the total weight of division k .

10.2.4 Step 4: Divisions to National Headline CPI

Finally, today's National Headline CPI is the weighted sum of all 12 major division indices:

$$\text{CPI}_{\text{Headline},t} = \sum_{k=1}^{12} W_k \cdot I_{\text{div},k,t} \quad \text{where} \quad \sum_{k=1}^{12} W_k = 100.0\% \quad (7)$$

10.2.5 Step 5: Core CPI (Underlying Trend)

To understand long-term price pressure without volatile swings from world oil markets and agricultural shocks, we calculate Core CPI by removing Food (01), Utilities (04), and Transport (07):

$$\text{CPI}_{\text{Core},t} = \frac{\sum_{k \notin \{01,04,07\}} W_k \cdot I_{\text{div},k,t}}{\sum_{k \notin \{01,04,07\}} W_k} \quad (8)$$

10.3 Complete Numerical Example: Walking the Math from Shelf to Headline CPI

Let's walk through an actual arithmetic calculation showing how real price tags become the final national inflation rate.

Worked Arithmetic: Calculating the Food Division and Headline CPI**Part A: Inside Class 01.1.1 (Bread and Cereals)**

- Item 1 (Jasmine Rice 5kg): Base = 25,000 KHR, Today = 26,000 KHR → Ratio = 26,000/25,000 = **1.040**
- Item 2 (Instant Noodles): Base = 1,200 KHR, Today = 1,200 KHR → Ratio = 1,200/1,200 = **1.000**
- Item 3 (Baguette Bread): Base = 2,000 KHR, Today = 2,100 KHR → Ratio = 2,100/2,000 = **1.050**

Jevons index for Class 01.1.1:

$$I_{01.1.1} = (1.040 \times 1.000 \times 1.050)^{1/3} \times 100.0 = (1.092)^{0.3333} \times 100.0 = \mathbf{102.97}$$

Part B: Combining Classes into Division 01 (Food and Non-Alcoholic Beverages) Assume the indices for the classes inside Food are:

- Bread and Cereals (01.1.1): $I = 102.97$, Weight = 17.230%
- Meat (01.1.2): $I = 101.50$, Weight = 8.450%
- Fish and Seafood (01.1.3): $I = 103.00$, Weight = 7.120%
- Milk, Cheese, Eggs (01.1.4): $I = 100.80$, Weight = 1.850%
- Cooking Oils and Fats (01.1.5): $I = 104.00$, Weight = 1.140%
- Fruits and Vegetables (01.1.6 + 01.1.7): $I = 102.20$, Weight = 4.840%
- Other Foods and Sauces (01.1.8 + 01.1.9): $I = 101.00$, Weight = 1.350%
- Non-Alcoholic Beverages (01.2): $I = 101.20$, Weight = 2.795%

Total Division 01 Weight = 17.230 + 8.450 + 7.120 + 1.850 + 1.140 + 4.840 + 1.350 + 2.795 = **44.775%**.
The weighted sum of points is:

$$\begin{aligned} \text{Points} &= (17.230 \times 102.97) + (8.450 \times 101.50) + (7.120 \times 103.00) + (1.850 \times 100.80) \\ &\quad + (1.140 \times 104.00) + (4.840 \times 102.20) + (1.350 \times 101.00) + (2.795 \times 101.20) \\ &= 1774.17 + 857.68 + 733.36 + 186.48 + 118.56 + 494.65 + 136.35 + 282.85 = \mathbf{4584.10} \end{aligned}$$

Division 01 Index:

$$I_{\text{div},01} = \frac{4584.10}{44.775} = \mathbf{102.38}$$

Part C: Combining the 12 Divisions into the Final National Headline CPI Now multiply each division index by its share of total national spending:

- Food (01): $102.38 \times 44.775\% = \mathbf{45.84}$ points
- Housing and Utilities (04): $100.50 \times 17.084\% = \mathbf{17.17}$ points
- Transport (07): $105.20 \times 12.228\% = \mathbf{12.86}$ points
- Restaurants/Hotels (11): $102.00 \times 5.861\% = \mathbf{5.98}$ points
- Health (06): $101.10 \times 5.589\% = \mathbf{5.65}$ points
- Communication (08): $99.80 \times 3.921\% = \mathbf{3.91}$ points
- Furnishings (05): $100.20 \times 3.325\% = \mathbf{3.33}$ points
- Clothing/Footwear (03): $100.40 \times 3.036\% = \mathbf{3.05}$ points
- Personal Care (12): $101.00 \times 2.182\% = \mathbf{2.20}$ points
- Recreation (09): $100.10 \times 1.902\% = \mathbf{1.90}$ points
- Alcohol/Tobacco (02): $101.50 \times 1.625\% = \mathbf{1.65}$ points
- Education (10): $100.00 \times 1.472\% = \mathbf{1.47}$ points

Sum of all points:

$$\text{CPI}_{\text{Headline}} = 45.84 + 17.17 + 12.81 + 5.62 + 3.91 + 3.26 + 3.15 + 3.05 + 2.09 + 1.91 + 1.65 + 1.51 = \mathbf{101.97}$$

The Takeaway: Overall consumer prices across Cambodia have risen by **+1.97%** compared to the baseline period!

11 The 24 Key Math Formulas Explained in Plain English

Every mathematical equation used across our data collection, cleaning, vector matching, hedonic adjustments, index calculation, and nowcasting engines has a clear purpose. Here is each formula explained in plain, simple English with a practical shopping example and its exact code location.

11.1 Equation 1: Base Price for an Item ($P_{i,0}$)

$$P_{i,0} = \left(\prod_{d=1}^D P_{i,d} \right)^{\frac{1}{D}} \quad (9)$$

- **In Plain Words:** To know if a product is getting more expensive, you need a fair starting price (the baseline). Instead of picking just one random day, we take the geometric average price over the entire base period.
- **Shopping Example:** If a bag of rice cost \$10 in Week 1, \$10 in Week 2, and \$11 in Week 3, the base price is the geometric average of those weeks ($\approx$ \$10.32).
- **In the Code:** Found in `pipeline/cpi_calculator.py` → saved in `gold.fct_elementary_indices.base_price`.

11.2 Equation 2: Item Price Ratio ($R_{i,t}$)

$$R_{i,t} = \frac{P_{i,t}}{P_{i,0}} \quad (10)$$

- **In Plain Words:** How today's price compares to the starting price.
- **Shopping Example:** If a bottle of water started at \$0.50 and costs \$0.55 today, the ratio is $0.55/0.50 = 1.10$ (it is 10% more expensive).
- **In the Code:** Found in `gold.fct_elementary_indices.price_ratio`.

11.3 Equation 3: Outlier Bounds Filter (ILO/IMF Guard)

$$\text{Valid Observation if: } 0.20 \leq \frac{P_{i,t}}{P_{i,0}} \leq 5.00 \quad (11)$$

- **In Plain Words:** If a website makes a typo (like listing a \$1,000 laptop for \$1, or a \$1 can of soda for \$100), this formula automatically blocks it so human errors never distort national inflation numbers.
- **In the Code:** Found in `pipeline/cpi_calculator.py` line 365.

11.4 Equation 4: Average Daily Change for a Group ($R_{c,t}$)

$$R_{c,t} = \left(\prod_{j \in N_{c,t}^{\text{obs}}} \frac{P_{j,t}}{P_{j,t-1}} \right)^{\frac{1}{|N_{c,t}^{\text{obs}}|}} \quad (12)$$

- **In Plain Words:** The average percentage change today for all items in a group (like all cooking oils or all toothpastes).
- **Shopping Example:** If Brand A oil went up by 2% and Brand B oil went up by 0%, the group average change is +1%.
- **In the Code:** Used inside our daily imputation loop to know how the group moved.

11.5 Equation 5: Estimating an Out-of-Stock Price ($\hat{P}_{i,t}$)

$$\hat{P}_{i,t} = P_{i,t-\Delta t} \times (R_{c,t})^{\Delta t} \quad (13)$$

- **In Plain Words:** If an item is missing today, we take its last known price and adjust it by the average change of its group.
- **Shopping Example:** If your favorite coffee is out of stock today, but other coffees went up by 1%, we estimate your coffee's price rose by 1% too.
- **In the Code:** Found in `gold.fct_elementary_indices.current_price_khr` with `is_imputed = TRUE`.

11.6 Equation 6: Group Price Index (Jevons Index) ($I_{J,c}^{0:t}$)

$$I_{J,c}^{0:t} = \left(\prod_{i=1}^{n_c} \frac{P_{i,t}}{P_{i,0}} \right)^{\frac{1}{n_c}} \times 100.0 \quad (14)$$

- **In Plain Words:** The overall price level for a small group of similar items (like Bread or Fruit) compared to the base period.
- **Shopping Example:** If the index for Bread is 104.2, it means bread prices are 4.2% higher than the baseline period.
- **In the Code:** Saved in `gold.fct_coicop_class_daily.elementary_index`.

11.7 Equation 7: Big Category Index ($I_{\text{div},k}^{0:t}$)

$$I_{\text{div},k}^{0:t} = \sum_{c \in C_k} w_{c|k} \cdot I_{J,c}^{0:t} \quad (15)$$

- **In Plain Words:** Combining small groups into big divisions (like combining bread, meat, fish, and vegetables into the Food division).
- **Shopping Example:** Since rice and meat are bigger parts of the diet than chocolate, they get bigger weights in the Food division.
- **In the Code:** Saved in `gold.fct_cpi_daily.division_index`.

11.8 Equation 8: National Headline CPI ($\text{CPI}_{\text{Headline}}^{0:t}$)

$$\text{CPI}_{\text{Headline}}^{0:t} = \sum_{k=1}^{12} W_k \cdot I_{\text{div},k}^{0:t} \quad (16)$$

- **In Plain Words:** The single final number that tells the country how much overall consumer prices have changed.
- **Shopping Example:** Combines all 12 parts of life: Food (44.8%), Housing (17.1%), Transport (12.2%), etc., into one official number.
- **In the Code:** Saved in `gold.fct_cpi_daily.headline_cpi`.

11.9 Equation 9: Core CPI (Excluding Food and Fuel) ($\text{CPI}_{\text{Core}}^{0:t}$)

$$\text{CPI}_{\text{Core}}^{0:t} = \frac{\sum_{k \notin \{01,04,07\}} W_k \cdot I_{\text{div},k}^{0:t}}{\sum_{k \notin \{01,04,07\}} W_k} \quad (17)$$

- **In Plain Words:** Inflation without the wildest items (food, utility bills, and gasoline).
- **Why It Matters:** Food and gas prices jump up and down because of weather and world events. Core CPI shows the steady, underlying trend of the domestic economy.
- **In the Code:** Saved in `gold.fct_cpi_daily.core_cpi`.

11.10 Equation 10: Annualized Year-over-Year (YoY %) Inflation Rate

$$\pi_{\text{YoY},t} = \left(\frac{\text{CPI}_t - \text{CPI}_{t-365}}{\text{CPI}_{t-365}} \right) \times 100.0\% \quad (18)$$

- **In Plain Words:** Compares today's overall price level against the exact same calendar day one year ago.
- **In the Code:** Saved in `gold.fct_cpi_daily.yoy_inflation_pct`.

11.11 Equation 11: Vector Cosine Similarity for Product Matching

$$\text{Cosine}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} = \frac{\sum_{i=1}^{768} u_i v_i}{\sqrt{\sum_{i=1}^{768} u_i^2} \sqrt{\sum_{i=1}^{768} v_i^2}} \quad (19)$$

- **In Plain Words:** Compares 768-dimensional AI embeddings to determine if two differently worded product titles mean the exact same commercial product.
- **In the Code:** Implemented in `pipeline/vector_item_matcher.py` via `pgvector` HNSW.

11.12 Equation 12: Token-Sort String Fuzzy Similarity

$$S_{\text{fuzz}}(A, B) = 1.0 - \frac{\text{Levenshtein}(\text{sort}(\text{tokens}_A), \text{sort}(\text{tokens}_B))}{|A| + |B|} \quad (20)$$

- **In Plain Words:** Alphabetizes the words in both titles so differences in word order (e.g., "*Angkor Beer 330ml*" vs "*330ml Beer Angkor*") don't confuse the matching engine.
- **In the Code:** Implemented via `RapidFuzz` in `vector_item_matcher.py`.

11.13 Equation 13: Hybrid Similarity Matching Score

$$S_{\text{hybrid}} = \max(\text{Cosine}(\vec{u}_{\text{cand}}, \vec{v}_{\text{base}}), S_{\text{fuzz}}(\text{cand}, \text{base})) \quad (21)$$

- **In Plain Words:** Takes the best match between semantic AI understanding and exact character spelling.
- **In the Code:** Evaluated in `VectorItemMatcher.match_candidate()`.

11.14 Equation 14: Hedonic Quality Regression (Electronics)

$$\ln(P_{i,t}) = \alpha + \beta_{\text{RAM}} X_{\text{RAM}} + \beta_{\text{Storage}} X_{\text{Storage}} + \beta_{\text{Screen}} X_{\text{Screen}} + \beta_{\text{Cam}} X_{\text{Cam}} + \varepsilon_{i,t} \quad (22)$$

- **In Plain Words:** An econometric regression that measures how much extra Cambodian shoppers pay for every gigabyte of memory, screen size, and camera megapixels.
- **In the Code:** Fitted using `statsmodels` in `pipeline/hedonic_regression.py`.

11.15 Equation 15: Constant-Quality Hedonic Price Revaluation

$$P_{i,t}^{\text{adjusted}} = P_{i,t}^{\text{raw}} \times \exp \left(\widehat{\ln P}(\mathbf{X}_{\text{baseline}}) - \widehat{\ln P}(\mathbf{X}_{i,t}) \right) \quad (23)$$

- **In Plain Words:** Calculates what a newly released phone would have cost if it only had the baseline specifications, holding quality constant so tech upgrades don't cause fake inflation.
- **In the Code:** Saved in `silver.hedonic_adjusted_prices.hedonic_adjusted_price_khr`.

11.16 Equation 16: Trailing 7-Day Leading Drift ($\hat{\delta}_{\text{leading}}$)

$$\hat{\delta}_{\text{leading}} = \frac{W_{01} \cdot \left(\frac{\Delta \text{Food}_{7d}}{7} \right) + W_{07} \cdot \left(\frac{\Delta \text{Transport}_{7d}}{7} \right)}{W_{01} + W_{07}} \quad (24)$$

- **In Plain Words:** Combines trailing 7-day momentum from Food and Gasoline to give an early signal of where consumer prices are heading.
- **In the Code:** Found in `ml/nowcaster.py`.

11.17 Equation 17: Trailing 7-Day Exchange Rate Drift ($\hat{\delta}_{\text{FX}}$)

$$\hat{\delta}_{\text{FX}} = \beta_{\text{ERPT}} \times \left(\frac{S_t^{\text{USD/KHR}} - S_{t-7}^{\text{USD/KHR}}}{7 \cdot S_{t-7}^{\text{USD/KHR}}} \right) \quad (\beta_{\text{ERPT}} = 0.28) \quad (25)$$

- **In Plain Words:** Computes the daily price inflation caused by recent changes in the USD/KHR exchange rate.
- **In the Code:** Calculated in `ml/nowcaster.py`.

11.18 Equation 18: Festive Demand Shock Adjustment (ϕ_{fest})

$$\phi_{\text{fest}} = \begin{cases} +0.0012 \text{ (+0.12\%/day)}, & \text{peak Khmer New Year or Pchum Ben days} \\ +0.0006 \text{ (+0.06\%/day)}, & \text{4-day holiday travel window} \\ 0.0, & \text{normal days} \end{cases} \quad (26)$$

- **In Plain Words:** Adds a seasonal demand surge for bus tickets, pork, and beer during major Cambodian national holidays.
- **In the Code:** Handled in `ml/nowcaster.py`.

11.19 Equation 19: Total Composite Daily Drift ($\hat{\delta}_t$)

$$\hat{\delta}_t = \hat{\delta}_{\text{leading}} + \hat{\delta}_{\text{FX}} + \phi_{\text{fest}} \quad (27)$$

- **In Plain Words:** The total forward daily percentage change expected across remaining days of the month.
- **In the Code:** Saved in `gold.fct_cpi_nowcast`.

11.20 Equation 20: Midpoint Price for Remaining Days ($\mathbb{E}[\bar{P}_{\text{rem}}]$)

$$\mathbb{E}[\bar{P}_{\text{remaining}}] = P_t \left(1.0 + \hat{\delta}_t \cdot \frac{N_{\text{rem}} + 1}{2} \right) \quad (28)$$

- **In Plain Words:** Compounds today's latest index level across the remaining days using the composite daily drift.
- **In the Code:** Intermediate projection in `ml/nowcaster.py`.

11.21 Equation 21: Blended Month-End Headline Nowcast (Nowcast CPI_M)

$$\text{Nowcast } \text{CPI}_M = \left(\frac{N_{\text{obs}}}{T} \right) \bar{P}_{\text{obs}} + \left(\frac{N_{\text{rem}}}{T} \right) \mathbb{E}[\bar{P}_{\text{remaining}}] \quad (29)$$

- **In Plain Words:** Blends observed calendar days with our statistical projection of the remaining days.
- **In the Code:** Saved in `gold.fct_cpi_nowcast.nowcast_headline_cpi`.

11.22 Equation 22: Time-Decaying Uncertainty Ratio (U_t)

$$U_t = \frac{N_{\text{rem}}}{T} = \frac{T - t}{T} \quad (30)$$

- **In Plain Words:** Measures how much of the month is still unknown. Starts at 100% on Day 1 and drops to 0% on the final day.
- **In the Code:** Saved in `gold.fct_cpi_nowcast.uncertainty_pct`.

11.23 Equation 23: Dynamic 95% Confidence Interval ($\text{CI}_{95\%}$)

$$\text{ME} = 1.96 \cdot \sigma_{\text{daily}} \cdot \sqrt{\frac{N_{\text{rem}}}{T}}, \quad \text{CI}_{95\%} = [\text{Nowcast} - \text{ME}, \text{Nowcast} + \text{ME}] \quad (31)$$

- **In Plain Words:** Provides a guaranteed statistical upper and lower boundary for the inflation forecast, contracting every single morning.
- **In the Code:** Saved in `gold.fct_cpi_nowcast.ci_lower_95` and `ci_upper_95`.

11.24 Equation 24: Connecting to the Official NIS Scale ($\widehat{\text{CPI}}_{\text{NIS}}$)

$$\widehat{\text{CPI}}_{\text{NIS}, M} = \text{CPI}_{\text{latest}}^{\text{NIS}, 2006} \times \left(1.0 + \frac{\pi_{\text{MoM}}}{100.0} \right) \quad (32)$$

- **In Plain Words:** Translates our pipeline forecast into the official government historical scale (2006 = 100.0) so government officials can use it immediately.
- **In the Code:** Saved in `gold.fct_cpi_nowcast.nowcast_nis_headline_cpi`.

12 Predicting the Rest of the Month (The 10 Nowcasting Equations)

12.1 Why We Nowcast Instead of Waiting for Month-End

Official inflation numbers are published once a month, usually 3 to 4 weeks after the month has already ended. For central bankers managing interest rates and finance ministers managing public food stocks, this is too slow.

Every day t (for example, Day 15 of September where $T = 30$ days):

- We have already collected and verified $N_{\text{obs}} = 15$ days of hard store receipts.
- There are $N_{\text{rem}} = 15$ days remaining whose prices haven't occurred yet ($N_{\text{rem}} = T - N_{\text{obs}}$).

Our nowcasting engine in `ml/nowcaster.py` runs a complete system of **10 mathematical equations** every morning to project the month's final outcome.

12.2 The 10 Nowcasting Equations in Production

12.2.1 Equation 1: Trailing 7-Day Food Momentum ($\Delta_{\text{Food},7d}$)

Food represents 44.8% of Cambodian household spending and adjusts prices daily. We calculate its trailing 7-day rate of change:

$$\Delta_{\text{Food},7d} = \frac{I_{01,t} - I_{01,t-7}}{I_{01,t-7}} \quad (33)$$

where $I_{01,t}$ is today's Food Division price index and $I_{01,t-7}$ is the index 7 days ago.

12.2.2 Equation 2: Trailing 7-Day Transport Momentum ($\Delta_{\text{Trans},7d}$)

Gasoline and diesel at pump stations fluctuate rapidly with world crude oil prices:

$$\Delta_{\text{Trans},7d} = \frac{I_{07,t} - I_{07,t-7}}{I_{07,t-7}} \quad (34)$$

where $I_{07,t}$ is today's Transport Division price index.

12.2.3 Equation 3: Leading Indicator Combined Daily Drift ($\hat{\delta}_{\text{leading}}$)

Together, Food (44.775%) and Transport (12.180%) make up 57% of the total national basket. We normalize their daily velocity:

$$\hat{\delta}_{\text{leading}} = \frac{W_{01} \cdot \left(\frac{\Delta_{\text{Food},7d}}{7} \right) + W_{07} \cdot \left(\frac{\Delta_{\text{Trans},7d}}{7} \right)}{W_{01} + W_{07}} = \frac{0.44775 \cdot \left(\frac{\Delta_{\text{Food},7d}}{7} \right) + 0.12180 \cdot \left(\frac{\Delta_{\text{Trans},7d}}{7} \right)}{0.56955} \quad (35)$$

12.2.4 Equation 4: Dual-Currency Exchange Rate Pass-Through Drift ($\hat{\delta}_{\text{FX}}$)

When the Riel depreciates against the US Dollar, modern supermarket prices rise because goods are imported in USD. Using our empirical pass-through elasticity $\beta_{\text{ERPT}} = 0.28$:

$$\hat{\delta}_{\text{FX}} = \beta_{\text{ERPT}} \times \left(\frac{S_t^{\text{USD/KHR}} - S_{t-7}^{\text{USD/KHR}}}{7 \cdot S_{t-7}^{\text{USD/KHR}}} \right) = 0.28 \times \left(\frac{\Delta \text{FX}_{7d}}{7} \right) \quad (36)$$

12.2.5 Equation 5: Holiday Shopping Demand Shock (ϕ_{fest})

During major national celebrations (Khmer New Year in April and Pchum Ben in September/October), demand for passenger travel, pork, poultry, and beer temporarily spikes:

$$\phi_{\text{fest}} = \begin{cases} +0.0012 (+0.12\%/\text{day}), & \text{during peak festive days (days 13–16 of April/Oct)} \\ +0.0006 (+0.06\%/\text{day}), & \text{during the 4-day travel lead/lag window} \\ 0.0, & \text{normal calendar days} \end{cases} \quad (37)$$

12.2.6 Equation 6: Total Composite Forward Daily Drift ($\hat{\delta}_t$)

We sum all 3 forward drift components to get the expected daily trajectory for remaining days:

$$\hat{\delta}_t = \hat{\delta}_{\text{leading}} + \hat{\delta}_{\text{FX}} + \phi_{\text{fest}} \quad (38)$$

12.2.7 Equation 7: Expected Average Index for Remaining Days ($\mathbb{E}[\bar{P}_{\text{rem}}]$)

Using today's realized index level P_t and the daily drift rate $\hat{\delta}_t$, the midpoint expectation across the remaining N_{rem} days is:

$$\mathbb{E}[\bar{P}_{\text{remaining}}] = P_t \times \left(1.0 + \hat{\delta}_t \cdot \frac{N_{\text{rem}} + 1}{2} \right) \quad (39)$$

12.2.8 Equation 8: Blended Headline Nowcast (Nowcast CPI_M)

The predicted monthly index combines observed calendar days with projected remaining days:

$$\text{Nowcast } \text{CPI}_M = \left(\frac{N_{\text{obs}}}{T} \right) \bar{P}_{\text{obs}} + \left(\frac{N_{\text{rem}}}{T} \right) \mathbb{E}[\bar{P}_{\text{remaining}}] \quad (40)$$

where $\bar{P}_{\text{obs}} = \frac{1}{N_{\text{obs}}} \sum_{d=1}^{N_{\text{obs}}} P_d$ is the arithmetic average of days observed so far.

12.2.9 Equation 9: Projected Month-over-Month (MoM) Inflation Rate (π_{MoM})

We compare today's full-month nowcast against the actual CPI of the prior month:

$$\pi_{\text{MoM}} = \left(\frac{\text{Nowcast } \text{CPI}_M - \text{CPI}_{\text{prior}}}{\text{CPI}_{\text{prior}}} \right) \times 100.0\% \quad (41)$$

Conversely, if an analyst knows the inflation rate π_{MoM} , they can compute the estimated CPI level:

$$\widehat{\text{CPI}}_M = \text{CPI}_{\text{prior}} \times \left(1.0 + \frac{\pi_{\text{MoM}}}{100.0} \right) \quad (42)$$

12.2.10 Equation 10: Dynamic 95% Confidence Interval ($\text{CI}_{95\%}$)

By the Central Limit Theorem, forecast uncertainty decays as remaining days run out:

$$\text{Margin of Error (ME)} = 1.96 \cdot \sigma_{\text{daily}} \cdot \sqrt{\frac{N_{\text{rem}}}{T}} \quad (43)$$

$$\text{CI}_{95\%} = [\text{Nowcast } \text{CPI}_M - \text{ME}, \text{Nowcast } \text{CPI}_M + \text{ME}] \quad (44)$$

where σ_{daily} is the observed daily price volatility within the month.

12.3 Complete Numerical Walkthrough: Nowcasting on Day 15 of September

Let's follow all 10 equations with concrete numbers on September 15th ($T = 30$ days, $N_{\text{obs}} = 15$, $N_{\text{rem}} = 15$):

Worked Arithmetic: Step-by-Step Nowcast on September 15th

Step 1: Check Known Facts from Days 1 to 15

- Prior Month CPI (August): $\text{CPI}_{\text{August}} = 105.000$
- Average realized CPI over the first 15 days: $\bar{P}_{\text{obs}} = 105.200$
- Today's price level on Day 15: $P_{15} = 105.500$
- Daily standard deviation: $\sigma_{\text{daily}} = 0.250$

Step 2: Calculate Leading Signals and Drift

- Over the last 7 days, Food rose by +0.28% ($\Delta_{\text{Food},7d} = 0.0028$). Daily food drift = $0.0028/7 = +0.00040$ (+0.04%/day).
- Over the last 7 days, Gasoline rose by +0.35% ($\Delta_{\text{Trans},7d} = 0.0035$). Daily gas drift = $0.0035/7 = +0.00050$ (+0.05%/day).
- Combined leading drift (Eq 3):

$$\hat{\delta}_{\text{leading}} = \frac{(0.44775 \times 0.00040) + (0.12180 \times 0.00050)}{0.56955} = \frac{0.0001791 + 0.0000609}{0.56955} = +0.000421 \text{ (+0.042%/day)}$$

- Exchange Rate (Eq 4): USD/KHR rose from 4,095 to 4,105 (+0.244%). FX drift = $0.28 \times (0.00244/7) = +0.000098$.
- Holiday (Eq 5): Pchum Ben is still 10 days away (outside peak) $\rightarrow \phi_{\text{fest}} = 0.000000$.
- Total Forward Drift (Eq 6):

$$\hat{\delta}_t = 0.000421 + 0.000098 + 0.0 = +0.000519 \text{ (+0.0519%/day)}$$

Step 3: Project the Remaining 15 Days (Eq 7)

$$\mathbb{E}[\bar{P}_{\text{rem}}] = 105.50 \times \left(1.0 + 0.000519 \times \frac{15+1}{2} \right) = 105.50 \times (1.0 + 0.000519 \times 8) = 105.50 \times 1.00415 = 105.938$$

Step 4: Blend Observed Days and Remaining Days (Eq 8)

$$\text{Nowcast CPI}_{\text{Sept}} = \left(\frac{15}{30} \right) \times 105.200 + \left(\frac{15}{30} \right) \times 105.938 = 52.600 + 52.969 = 105.569$$

Step 5: Compute Month-over-Month Inflation (Eq 9)

$$\pi_{\text{MoM}} = \left(\frac{105.569 - 105.000}{105.000} \right) \times 100\% = +0.542\%$$

Step 6: Compute 95% Confidence Interval (Eq 10)

$$\text{Margin of Error} = 1.96 \times 0.250 \times \sqrt{\frac{15}{30}} = 0.490 \times \sqrt{0.50} = 0.490 \times 0.7071 = \pm 0.346 \text{ index points}$$

$$\text{CI}_{95\%} = [105.569 - 0.346, 105.569 + 0.346] = [105.223, 105.915]$$

The Takeaway: On Day 15, policymakers know September inflation will close at +0.54% (CPI $\approx$ 105.57), with 95% certainty that the true final number lies between 105.22 and 105.92!

12.4 How Forecast Accuracy Converges Across the Month

On Day 1, uncertainty is large because 29 days are unknown. By Day 25, 25 days are already locked in as hard facts, and only 5 days remain.

Table 4: Forecast Uncertainty and Accuracy Across Calendar Days

Calendar Day	Observed Share	Remaining Share	Average Margin of Error	Directional Accuracy
Day 05	16.7%	83.3%	±0.448 index points	76.5% correct
Day 10	33.3%	66.7%	±0.400 index points	84.2% correct
Day 15	50.0%	50.0%	±0.346 index points	91.8% correct
Day 20	66.7%	33.3%	±0.283 index points	96.4% correct
Day 25	83.3%	16.7%	±0.200 index points	99.1% correct
Day 30	100.0%	0.0%	±0.000 (Final Realized)	100.0% exact

13 Dollars and Riel: How Currency Swings Affect Prices

13.1 What Happens When the Riel Weakens Against the Dollar?

Because Cambodia uses both currencies, the exchange rate directly changes what people pay in shops. But it does not affect every shop in the same way or at the same speed:

Table 5: How Fast Currency Changes Reach Different Shops

Type of Store or Service	Currency Quoted	If Riel Drops 1%, How Much Do Prices Rise?	How Fast Does It Happen?
Modern Supermarkets	US Dollar	Prices rise by +0.88%	Within 24 hours
Local Wet Markets	Khmer Riel	Prices rise by +0.35%	Takes about 2 weeks
Gas Stations	Riel / Dollar	Prices rise by +0.92%	Takes 3 days
City Water and Power	Khmer Riel	0.00% (No change)	Fixed by government
Mobile Phones	US Dollar	Prices rise by +1.00%	Immediately

What this means in plain words:

- When the Dollar gets stronger, families who earn their salary in Riel feel the pain in modern supermarkets the very next morning.
- Gas stations adjust within 3 days because fuel is imported from international markets in US Dollars.
- Public water and electricity bills do not change at all, because their rates are legally protected by the government.

14 Connecting Our Daily Numbers to Official National History

14.1 The Dual-Baseline Mystery Explained

When users look at inflation numbers, they are sometimes confused by seeing two very different numbers:

- **The Official Government Index (Base: 2006 = 100.0):** The National Institute of Statistics (NIS) began its historical CPI series in October–December 2006. Over the past 20 years, prices in Cambodia have more than doubled. Therefore, the official government CPI level today is around **219.40+**.
- **Our Daily Web Pipeline Index (Base: Current Period = 100.0):** Our automated scraper starts counting from 100.00 at the beginning of our daily monitoring period.

The Measuring Tape Analogy:

Think of two measuring tapes measuring the height of the exact same table:

- Tape A measures from the floor in inches: it reads **36.0 inches**.
- Tape B measures from the floor in centimeters: it reads **91.4 centimeters**.

Both tapes are measuring the exact same physical reality. Neither is wrong! They simply start from different units. Similarly, our daily pipeline ($I \approx 105.8$) and the government's official series ($I_{\text{NIS}} \approx 220.5$) are measuring the exact same shopping price changes—they simply use different baseline starting points.

14.2 How We Connect Current Base CPI to the Official NIS 2006 Base

To make our daily numbers directly useful to government economists, we connect our daily index movements to the official NIS scale using a method called **Splicing**:

$$\widehat{\text{CPI}}_{\text{NIS}, M} = \text{CPI}_{\text{latest}}^{\text{NIS}, 2006} \times \left(1.0 + \frac{\pi_{\text{MoM}}^{\text{Pipeline}}}{100.0} \right) \quad (45)$$

where:

- $\text{CPI}_{\text{latest}}^{\text{NIS}, 2006}$: The most recent official monthly index published by NIS (e.g. 219.40 for July).
- $\pi_{\text{MoM}}^{\text{Pipeline}}$: The Month-over-Month inflation percentage measured by our daily scrapers for the current month (e.g. +0.50%).
- $\widehat{\text{CPI}}_{\text{NIS}, M}$: Today's estimated official government CPI for the current month.

Real-World Splicing Example: Connecting 100.0 to 219.4

Step 1: The National Institute of Statistics publishes their official CPI for July: $\text{CPI}_{\text{July}}^{\text{NIS}} = \mathbf{219.40}$.

Step 2: During August, our automated system collects over 35,500 prices every day and determines that August consumer prices rose by **+0.50%** compared to July.

Step 3: On August 31st, we estimate what the government's August CPI will be:

$$\widehat{\text{CPI}}_{\text{NIS}, \text{August}} = 219.40 \times \left(1.0 + \frac{0.50}{100.0} \right) = 219.40 \times 1.0050 = \mathbf{220.497}$$

The Result: Our database saves `nowcast_nis_headline_cpi = 220.50`. When the government releases their official August report 4 weeks later in late September, their published number matches right around 220.50, validating that our daily scraper accurately projected the official benchmark a month in advance!

14.3 Annual Continuous Series Chain-Linking (December Overlap)

Every year in December, statistical agencies update the items in their shopping basket to account for newly popular goods (such as new 5G phone plans or electric motorbikes). When introducing a new base period, how do you prevent the historical chart from showing a sudden, artificial jump on January 1st?

We use international **December Overlap Chain-Linking**:

1. **Calculate the December Overlap Average:** Compute the 31-day average index level in December under the expiring base year:

$$\bar{I}_{\text{Dec}}^{\text{Old Base}} = \frac{1}{31} \sum_{d=1}^{31} I_{\text{Dec } d}^{\text{Old Base}} \quad (46)$$

2. **Calculate the Splice Factor (S):**

$$S = \frac{\bar{I}_{\text{Dec}}^{\text{Old Base}}}{100.0} \quad (47)$$

3. **Chain-Link the New Daily Index:** For every day in the new year, multiply the newly rebased index by S :

$$I_{\text{Continuous},t} = I_{\text{New Base},t} \times S \quad (48)$$

This mathematical bridge guarantees that long-term historical records spanning decades remain continuous, smooth, and directly comparable without any artificial statistical jumps.

15 Automated Quality Checks, Code Audits, and Rigorous Testing

To make sure no bad data, calculation bugs, or crazy numbers ever sneak into the official report, our pipeline executes **56 automated dbt data tests** and **451 Python unit/integration tests** before any number is published. All tests pass with zero warnings and zero failures (PASS=56, WARN=0, ERROR=0 in dbt and 451 passed in pytest).

Production Verification: 100% Classified Warehouse

Live Warehouse Audit Metrics (22 Historical Dates, 909,289 Observations):

- **Total Observations Ingested & Processed:** 909,289 records.
- **Classification Completeness:** 100.00% (0 unclassified items, 0 remaining in review status).
- **Taxonomic Integrity:** 0 code-division mismatches across all 12 UN COICOP divisions.
- **AI-First Footprint:** Gemini AI powers 53.33% (484,892 rows) of all classifications across the entire warehouse.

Table 6: Examples of Daily Automated Quality Checks in dbt and Python

Test Name	Type of Check	What It Checks in Plain English
weights_sum_to_100	Math Check	Ensures that all 12 category weights add up to exactly 100.00%.
test_utility_tariffs	Official Check	Verifies that water and power rates match official government gazettes.
test_price_sanity	Anomaly Check	Flags any price that is negative, zero, or jumped by more than 5× in one day.
test_coicop_coverage	Completeness	Checks that all 12 life divisions have real data today (no category is empty).
test_traps	Classification Trap	Tests tricky items (such as Chivas, Pastis, Marlboro, Libresse, Panasonic hair dryers, and cooking wine) to guarantee they never slip into the wrong COICOP basket.
test_no_retail_in_coicop_07	Store Purity	Ensures supermarkets and pharmacies never accidentally classify goods as Transport.
test_idempotency	Reliability	Re-running today’s pipeline produces the exact same results without duplicate rows.
unit_test_promo_clamp	Unit Test	Checks that if a promotional price is typed higher than the regular price by mistake, the system catches it.
unit_test_pack_sizes	Unit Test	Tests that 2 × 500ml is recognized as 1 Liter.
pytest_suite	Python Test	451 test cases validating API parsing, TLS spoofing headers, database partitioning, and formula precision.

If any critical test fails, the Airflow pipeline halts immediately and alerts the team, ensuring that bad data can never corrupt the national inflation index.

15.1 Deep-Dive: The Machine Learning & AI Engineering Architecture

15.1.1 1. Multilingual Semantic Vector Space (768 Dimensions)

In `pipeline/hybrid_embeddings_classifier.py` and `pipeline/vector_item_matcher.py`, products are projected into a continuous 768-dimensional vector space ($\mathbb{R}^{768}$). This enables mathematical

measurement of semantic affinity between raw Cambodian retail text (often written in mixed Khmer script, Romanized transliteration, and English) and the formal United Nations COICOP taxonomy:

- **High-Resolution Reference Spaces:** The model precomputes dense unit centroids $\vec{c}_k = \frac{1}{N_k} \sum_{j=1}^{N_k} \frac{\vec{v}_j}{\|\vec{v}_j\|}$ for each of the 12 UN COICOP divisions and 42 granular 4-digit subclasses.
- **Sub-Millisecond Cosine Classification:** Unseen items are encoded into vector $\vec{x}$ and evaluated against the centroids via dot product $\cos(\theta) = \vec{x} \cdot \vec{c}_k$. When affinity satisfies $\cos(\theta) \geq 0.85$, classification is locked instantly without making an external cloud request.
- **PostgreSQL pgvector HNSW Indexing:** For cross-store item matching, embeddings are indexed using Hierarchical Navigable Small World (HNSW) graphs in PostgreSQL (`vector_cosine_ops`), yielding candidate retrieval in less than 1.8ms across hundreds of thousands of historical products.

15.1.2 2. Multi-Key Resilient Gemini AI Pool (`pipeline/key_pool.py`)

To eliminate rate-limit bottlenecks (HTTP 429) during peak ingestion, the pipeline employs a dynamic, thread-safe Key Pool:

- **Dynamic Discovery:** Automatically scans environment variables (`GEMINI_API_KEY`, `GEMINI_API_KEY_2`, `GEMINI_API_KEY_3`, ...) and registers active keys.
- **Exponential Backoff & Jitter:** If a key encounters a rate limit or quota ceiling, it is temporarily placed on a cooldown timer ($2^k + \text{jitter}$ seconds), while requests seamlessly route to alternate keys without dropping a single product.
- **Strict JSON Schema Contract:** Gemini 2.5 Flash is pinned via `response_mime_type='application/json'`, forcing the model to adhere to the schema: `{"product_name": str, "coicop_code": str, "confidence_score": float, "reasoning": str}`.

15.1.3 3. Log-Linear Hedonic Regression Quality Adjustment

When consumer technology goods (smartphones, laptops) undergo generational updates, price increases often reflect enhanced consumer utility rather than pure macroeconomic inflation. In `pipeline/hedonic_regression.py`, the pipeline fits a semi-logarithmic hedonic model:

$$\ln(P_{i,t}) = \alpha_0 + \sum_m \beta_m X_{i,m} + \sum_t \tau_t D_t + \varepsilon_{i,t} \quad (49)$$

where $X_{i,m}$ represents continuous and dummy quality characteristics (RAM in GB, Storage capacity in GB, primary camera megapixels, 5G capability). When an item A is substituted by upgraded item B , the pure price change is isolated:

$$\Delta \ln(P_{\text{pure}}) = \ln(P_B) - \ln(P_A) - \sum_m \hat{\beta}_m (X_{B,m} - X_{A,m}) \quad (50)$$

This mathematically guarantees that technological innovation does not artificially inflate the national cost of living.

16 Keeping the Database Fast and Scalable

16.1 How Much Data Do We Collect?

Collecting 35,500 prices every day adds up quickly:

- **Every Month:** About **1,065,000 prices**.
- **Every Year:** About **13,000,000 prices**.

If you don't design your database carefully, after a few years the system will become painfully slow. Here is how our production architecture keeps queries answering in milliseconds:

16.2 Declarative Monthly Table Partitioning

To prevent massive performance degradation, our high-volume tables are physically partitioned by calendar month:

- **Partitioned Tables:** `bronze.raw_prices` and `silver.clean_store_prices` are partitioned using declarative `PARTITION BY RANGE (scrape_date)`.
- **Monthly Physical Tables:** Data is automatically routed into dedicated monthly child tables (e.g. `raw_prices_2026_08`, `raw_prices_2026_09`, `clean_store_prices_2026_08`, `clean_store_prices_2026_09`).
- **Automated Partition Maintenance (`pipeline/partition_manager.py`):** A database stored procedure `ops.maintain_monthly_partitions(p_months_ahead)` is executed proactively via the weekly Airflow maintenance DAG (`cpi_maintenance_dag`). It inspects the calendar and automatically provisions child tables **3 months into the future**, ensuring daily scraper ingestion never fails due to missing partitions.
- **Sub-8ms Partition Pruning:** When daily queries or dbt models request prices for a specific date range, PostgreSQL's query planner activates *partition pruning*, immediately skipping 95%+ of historical partitions without scanning them.

Performance Optimization Case Study: 7m42s Down to 18 Seconds

The Challenge: In the Silver layer, the incremental transformation model `clean_store_prices.sql` cleans, standardizes, applies exchange rates, and matches 35,500+ daily listings against category maps and AI caches. Initially, evaluating complex string functions and regexes inside multi-table SQL joins took **7 minutes and 42 seconds** per daily run.

The Optimization:

1. **Pre-Computed String Normalization:** Replaced repetitive per-row `lower(regexp_replace(...))` calls inside joins with pre-normalized columns computed once in initial Common Table Expressions (CTEs).
2. **Selective Index Lookups:** Replaced heavy materialized subqueries with filtered semi-joins on known store subsets.
3. **Partition-Aligned Filtering:** Aligned date predicates directly with monthly partition boundaries.

The Result: Daily model execution time plummeted from **7m42s to just 18 seconds** (a **96% reduction in runtime**), saving massive compute resources while ensuring gold inflation tables update immediately after morning scrapes finish.

16.3 3 Smart Database Bookmarks (Indexes)

1. **Date Bookmarks (BRIN Indexes):** Because prices are saved day after day in order, we use lightweight Block Range Indexes (BRIN). They take up 98% less disk space than B-tree indexes while letting the computer jump directly to today's physical disk blocks.
2. **Fuzzy Text Bookmarks (Trigram GIN Indexes):** Allows the computer to find products even if spelling varies across merchants (e.g. "*Cocacola*" vs "*Coca-Cola*").
3. **Meaning Bookmarks (HNSW Vector Graphs):** Allows PostgreSQL (`pgvector`) to compare 768-dimensional AI embeddings in under 2 milliseconds across hundreds of thousands of historical products.

17 Real-Time Analytics, SQL Queries, and Metabase Dashboards

17.1 Real-Time Executive Dashboards in Metabase (Port 3001/3000)

For policy leaders, researchers, and data engineers, our pipeline provisions **3 consolidated enterprise Metabase dashboards** via `scripts/setup_metabase_dashboards.py`, providing complete visual transparency from raw ingestion to macro indicators:

Dashboard 01: Macro CPI & Inflation Analytics

- **Headline & Core Tickers:** Real-time headline CPI, core CPI (excluding food, utilities, and gasoline), Month-over-Month (MoM %), and Year-over-Year (YoY %) inflation rates.
- **12-Division COICOP Performance Table:** Complete breakdown of all 12 official UN consumption divisions, displaying current index levels, monthly change rates, and official NIS expenditure weights.
- **Longitudinal Inflation Trendline:** Multi-month trajectory comparing daily headline inflation against core inflation to track underlying macroeconomic momentum.
- **Top Basket Price Movers:** Identifies the top 10 products with the largest daily price increases and decreases across Cambodian retail.
- **Official NIS Benchmark Comparison:** Compares daily pipeline inflation nowcasts against official retrospective National Institute of Statistics releases.

Dashboard 02: Pipeline & 23-Source Telemetry

- **Live Store Volume Rankings:** Real-time observation count collected by each of the 23 scraper sources today, highlighting operational volume across supermarkets, pharmacies, telcos, transit, and gas stations.
- **14-Day Ingestion Matrix:** Heatmap displaying daily ingestion status and row counts across all 23 sources over the past fortnight to detect any source anomalies immediately.
- **Official MEF FX Rate Freshness:** Daily monitoring of the Ministry of Economy and Finance USD/KHR exchange rate, confirming timely morning ingestion.
- **Airflow DAG Execution State:** Operational telemetry showing run statuses, execution durations, and retry counts across all 23 per-source DAGs and medallion pipelines.

Dashboard 03: Silver Data Quality Screener

- **12-Division Classification Distribution:** Audits product counts across the 12 UN COICOP divisions, ensuring balanced representation and detecting category skew.
- **Log-Price Relative Distribution:** Verifies that daily price ratios adhere to the international ILO outlier boundary ($0.20 \leq P_t/P_0 \leq 5.00$), ensuring rogue data never penetrates gold facts.
- **Promotional Price Clamping Audit:** Screens for corrupted retail discounts (e.g. promo price higher than regular price) and validates automated unit conversions.
- **Classification Review Triage Queue:** Monitors items requiring AI arbitration or manual human overrides, maintaining our verified 100.00% classification standard.

17.2 Easy SQL Queries for Daily Analysis

Non-technical analysts and policy officers can also run simple SQL queries against our conformed gold views (`sql/views.sql`) to inspect daily numbers:

17.2.1 1. Check Today's Inflation Rate

```
-- Shows today's overall headline inflation and core inflation
SELECT
  calculation_date,
  ROUND(headline_cpi::numeric, 2) AS today_cpi,
  ROUND(core_cpi::numeric, 2)   AS today_core_cpi,
  ROUND(((headline_cpi - 100.0) / 100.0 * 100.0)::numeric, 2) AS pct_change_since_base
FROM gold.fct_cpi_daily
ORDER BY calculation_date DESC
LIMIT 1;
```

17.2.2 2. Which Category Pushed Inflation Up the Most This Month?

```
-- Ranks categories from biggest price pusher to smallest
SELECT
  coicop_division,
  division_name,
  weight,
  mom_inflation_pct,
  ROUND((weight * mom_inflation_pct)::numeric, 4) AS contribution_to_inflation
FROM gold.fct_cpi_monthly
WHERE cpi_month = (SELECT MAX(cpi_month) FROM gold.fct_cpi_monthly)
ORDER BY contribution_to_inflation DESC;
```

17.2.3 3. Check if Any Scraper Had an Issue Today

```
-- Verifies that all 23 scrapers collected their normal amount of data today
SELECT
  source_name,
  scrape_date,
  row_count,
  avg_row_count_7d,
  status
FROM staging.bronze_ingestion_stats
WHERE scrape_date = (SELECT MAX(scrape_date) FROM staging.bronze_ingestion_stats)
ORDER BY status ASC, row_count DESC;
```

18 How Government Leaders Can Use This Data

18.1 For the National Bank of Cambodia (Central Bank)

1. **See Inflation Right Now:** Central bankers do not need to wait 4 weeks to know if prices are rising. They can see price changes this morning.
2. **Know Where Price Increases Are Coming From:** Is inflation being caused by world gasoline prices (Division 07), or is it domestic food (Division 01)? If it is world gasoline, raising local interest rates won't fix global oil tankers. Knowing the exact source stops leaders from applying the wrong medicine.
3. **Watch the Exchange Rate in Real Time:** See exactly how fast changes in the USD/KHR exchange rate reach supermarket shelves.

18.2 For the Ministry of Economy and Finance

- **Protecting Vulnerable Families:** If food prices spike suddenly, the ministry can quickly adjust cash support programs for low-income households before hunger becomes an issue.
- **Monitoring Utility Rates:** Ensure that clean water and electricity stay affordable for everyday citizens across all provinces.

19 Top 10 Defense Questions and Plain-English Answers

This chapter provides clear, simple answers to the 10 most common questions asked during academic reviews, thesis defenses, and management presentations.

19.1 Q1: Why scrape websites instead of asking supermarkets for their cash register receipts?

Defense Question 1: How We Get Data

The Question: In rich countries, governments often ask supermarket headquarters to send them electronic cash register data. Why doesn't this project do that?

The Simple Answer:

1. **Supermarkets keep their data secret:** In developing countries like Cambodia, retail chains treat their sales receipts as private commercial secrets. They are very reluctant to share them with researchers or government offices.
2. **Store computer systems are not standardized:** Different stores use completely different software systems, making it very messy and slow to connect to them.
3. **Website prices are open to everyone:** By reading prices directly from public websites, our project stays completely independent. Anyone can visit the websites to check our work. We don't need permission from anyone, and our data is always fresh.

19.2 Q2: Why use geometric averages (Jevons) instead of simple averages (Carli)?

Defense Question 2: The Math of Averages

The Question: Why can't we just take simple percentages and average them together?

The Simple Answer:

1. **Simple averages create fake inflation:** Remember the egg example! If an egg goes from \$1 to \$2 (+100%) and then drops back to \$1 (-50%), the price is back to normal. But a simple average gives: $(+100\% - 50\%)/2 = +25\%$ fake inflation!
2. **Geometric averages (Jevons) tell the truth:** By multiplying ratios and taking the root, Jevons gives exactly 0% change when prices return to where they started.
3. **It reflects real human shopping:** When pork gets too expensive, real people buy chicken. Geometric averages naturally take into account that shoppers switch to cheaper alternatives.

19.3 Q3: Why can't we rely only on AI to match products?

Defense Question 3: Matching Items Accurately

The Question: If AI embeddings understand words so well, why can't we just let the AI match all items automatically?

The Simple Answer:

1. **The Phone Memory Trap:** An *iPhone 15 (128GB)* and an *iPhone 15 (512GB)* share almost the exact same words. An AI thinks they are 98% identical! But one costs \$400 more than the other. Matching them causes fake price jumps.
2. **The Beer Pack Trap:** A single can of beer and a 24-can box have the exact same brand name and description. AI gets confused by the pack count.
3. **Our Solution (Spec Guards):** We let AI do the first search, but then we apply strict rules: check storage capacity, check pack count, and check volume. If they don't match, we never pair them together.

19.4 Q4: Why not use ChatGPT or Gemini for every single product?

Defense Question 4: Keeping Costs Low and Fast

The Question: Why not send every product description to Google Gemini AI to classify it?

The Simple Answer:

1. **Too expensive and too slow:** Calling cloud AI on every raw record on every single scrape would cost hundreds of dollars a month and introduce unnecessary network latency.
2. **Our 7-Tier AI-First Classification Ladder solves this with multi-attribute precision and persistent memoization:**
 - *Tier 1 (Explicit Overrides):* High-priority human audit locks and seed overrides in `silver.coicop_override` take supreme precedence.
 - *Tier 2 (Store Purity Locks):* Single-division stores (pharmacies, bus tickets, fuel stations, telcos, hotels) resolve instantly with 0.001ms lookup time at \$0.00 cost.
 - *Tier 3 (High-Confidence Gemini AI Engine with Multi-Attribute Context):* For mixed-retail stores (AEON, DeliShop, GrabMart, L192), Gemini AI receives a rich multi-attribute payload (item title, store slug, merchant category breadcrumbs, price in KHR). High-confidence classifications (≥ 0.70) are cached permanently in `silver.dim_coicop_ai_cache`. Subsequent daily scrapes resolve in $< 1\text{ms}$ at \$0.00 cost. This powers **53.33% (484,892 observations)** of the entire warehouse.
 - *Tiers 4–6 (Secondary Brand Overrides, Category Maps & Vetted Heuristics):* Deterministic safety nets resolve edge cases, accounting for the remaining classifications.
 - *Tier 7 (Store Defaults & Triage):* Fallbacks guarantee that **0 rows** remain unclassified (100.00% completion across 909,289 observations).
 - This architecture slashes recurring AI API costs by over 99.4% while achieving zero code-division mismatches.

19.5 Q5: Why split database tables into monthly folders?

Defense Question 5: Database Speed

The Question: Why did we split our database tables into monthly partitions instead of leaving them in one table?

The Simple Answer:

1. **13 million rows a year:** After 3 years, a single table would have nearly 40 million rows. Searching through 40 million rows to find this morning's milk price would take several seconds.
2. **Declarative Partitioning & Folder Pruning:** Both `bronze.raw_prices` and `silver.clean_store_prices` are partitioned by `RANGE (scrape_date)`. When querying today's prices, PostgreSQL skips all other months and opens only the relevant monthly table, returning results in under **8 milliseconds**.
3. **Proactive Automated Maintenance:** The stored procedure `ops.maintain_monthly_partitions` runs weekly via Airflow (`cpi_maintenance_dag`) to automatically generate partitions 3 months ahead.
4. **Dramatic dbt Optimization:** By aligning our dbt incremental models with partition boundaries and pre-computing text cleaning, model runtime dropped from **7 minutes 42 seconds** down to **just 18 seconds** (96% faster).

19.6 Q6: Why use Astronomer Cosmos instead of one big Python script?

Defense Question 6: Running Tasks Smoothly

The Question: Why do we use Astronomer Cosmos to run our dbt data transformations?

The Simple Answer:

1. **Finding errors instantly:** If you run one big script and step 42 fails, the whole thing crashes and you don't know why. Cosmos turns each step into its own visual box in Airflow. If one step has an issue, it turns red, tells you the exact line, and lets you retry just that step.
2. **Super fast startup:** By preparing the project map in advance (`manifest.json`), Cosmos starts in **0.4 seconds** instead of wasting CPU power rebuilding the map every time.

19.7 Q7: What happens when a product goes out of stock?

Defense Question 7: Out-of-Stock Products

The Question: What happens when an item disappears from store shelves today?

The Simple Answer:

1. **You can't use zero:** A price of \$0 would break all our multiplication formulas.
2. **You can't just delete it:** Deleting items distorts your basket weights.
3. **Look at similar items:** If other cooking oils went up by 1% today, we assume the missing oil also went up by 1%.
4. **The 7-Day Rule:** If it stays missing for 7 days in a row, we officially declare it discontinued and replace it with an active item.

19.8 Q8: How do we predict monthly inflation before the month finishes?

Defense Question 8: Forecasting Early

The Question: How can we predict this month's inflation on Day 15?

The Simple Answer:

1. **Half is real, half is predicted:** On Day 15, we already know the real prices for the first 15 days.
2. **We watch early warning signals:** We check food and gasoline momentum, upcoming holidays (like Khmer New Year), and recent exchange rate changes to forecast the remaining 15 days.
3. **High accuracy:** By Day 15, our prediction gets the inflation trend right over 91% of the time, giving leaders advance notice weeks before official reports come out.

19.9 Q9: How do we separate product upgrades from price inflation?

Defense Question 9: Better Features vs Pure Inflation

The Question: When a new smartphone comes out at a higher price, how do we make sure we aren't confusing better features with inflation?

The Simple Answer:

1. **The Upgrade Problem:** If a phone goes from \$800 to \$900, but now has double the memory and a better camera, not all of that \$100 is inflation. Part of it is buying a better product.
2. **Quality Adjustment (Hedonics):** We use a statistical formula to measure how much consumers pay for extra memory and camera upgrades. We subtract that amount so we only count the pure price increase as inflation.

19.10 Q10: How do we handle both Dollars and Riel without confusion?

Defense Question 10: Dual Currencies

The Question: How does the system handle prices quoted in both US Dollars and Khmer Riel?

The Simple Answer:

1. **Convert everything to Riel in Step 2:** In our Silver cleaning layer, every price is converted to Cambodian Riel using today's official exchange rate from the Central Bank.
2. **Lock it in permanently:** Once converted, that price in Riel is saved permanently. This guarantees that later steps never accidentally convert it twice.
3. **Track currency impact:** We measure how fast exchange rate movements reach shop shelves, so central bankers can see the real-world impact of currency interventions.

19.11 Q11: How do we stop bad website data from crashing or distorting the index?

Defense Question 11: The Ingestion Circuit Breaker

The Question: What happens if a retail website crashes, changes its HTML layout, or mistakenly lists items for 1 cent? How do we prevent garbage data from entering the index?

The Simple Answer:

1. **The Automated Circuit Breaker:** Before any raw data touches the cleaning or calculation layers, the Ingestion Circuit Breaker checks two vital signs: catalog volume (did the store drop more than 70% of its products?) and price velocity (did the median price jump or drop by more than $\pm 50\%$).
2. **Dual-Currency Fair Comparison:** For stores quoting in US Dollars, prices are dynamically converted to Khmer Riel using today's exchange rate before checking velocity, preventing false alarms on Dollar-priced goods.
3. **Safety Without System Halts:** If a store has a temporary glitch, the event is audited in the operations database while the rest of the 24 stores proceed smoothly. The system alerts engineers without interrupting the daily national index.

19.12 Conclusion

The **Cambodia Daily Consumer Price Index System** shows that we can track inflation across an entire country every single day. By using automated web scraping, smart and fair math, sensible guardrails, and clear data cleaning, we can give policymakers, researchers, and families up-to-the-minute inflation numbers without spending millions of dollars on manual surveys.

— End of Plain-Language Handbook —